

Cover page

Declaration of Originality

Proforma

Contents

1	Introduction	7
1.1	Tagged Memory Architectures	7
1.2	Question 1 – Existing Mitigations	7
1.3	Question 2 – Potential Areas for Improvement	7
1.4	My Project	8
2	Preparation	9
2.1	Background Material	9
2.1.1	Tagged Memory Architectures	9
2.1.2	CHERI	10
2.1.3	A Basic Scheme	11
2.1.4	The Importance of a Tag Cache	13
2.1.5	<i>Efficient Tagged Memory</i> ’s Hierarchical Scheme	14
2.1.6	The Morello Project Scheme	18
2.2	Three-Stage Simulation	18
2.2.1	Workload Execution – LLC Miss Simulation	19
2.2.2	A Tag Controller Simulator	19
2.2.3	ChampSim – An Appropriate Cache Simulator	19
2.2.4	Visualisation Tool	20
2.2.5	Requirements Analysis	20
2.3	Starting Point	20
3	Implementation	22
3.1	Simulator	22
3.1.1	Simulator Framework	22
3.1.2	Trace Decoder	23
3.1.3	Simulated Memory	27
3.1.4	Tag Controller	29
3.1.4.1	Flat Table Controllers	30
3.1.4.2	Basic Scheme	31
3.1.4.3	<i>Efficient Tagged Memory</i> ’s Hierarchical Scheme	32
3.1.4.4	The Morello Project Scheme	34
3.2	Logging	36
3.2.1	Logging Tag Table State	36
3.2.2	Adding Logging to ChampSim for Tag Cache State	37
3.3	Verification	37
3.3.1	Microbenchmarks – Unit Tests	37
3.3.2	Macrobenchmark – Trace Verifier	37
3.3.3	Macrobenchmark – Cross-Verifier	38
3.4	Visualisation Tool	39
3.5	Phoenix – A Novel Scheme	41
3.5.1	The Motivation	41
3.5.2	The Intuition	41
3.5.2.1	Reference Counters	41
3.5.2.2	The Superroot Table	43
3.5.2.3	The Tag Working Set	44
3.5.3	The Algorithm	46

3.5.4	The Implementation	50
3.6	Repository Overview	51
4	Evaluation	52
5	Conclusions	53

List of Figures

1	How the two parts of a tagged value are stored in physical memory.	9
2	Step-by-step memory access translation.	10
3	How the parts of the memory system interface.	10
4	A CHERI capability.	11
5	The basic algorithm for reading tags. Memory access operations are blue.	11
6	The basic algorithm for writing tags. Operations internal to the tag controller are grey.	12
7	Where the tag cache sits in the memory system.	13
8	<i>Efficient Tagged Memory</i> 's two-level hierarchical lookup table.	14
9	The ETM algorithm for reading tags.	15
10	The ETM algorithm for writing tags.	16
11	The physical layout of the hierarchical table.	17
12	The layout of the two tag caches in the Morello memory system.	18
13	A black-box diagram explaining the three stage simulation.	19
14	An explanation of how the three main parts of the tag controller simulator interact with the inputs and outputs, and with each other.	22
15	The layout of an <code>llcMiss</code> struct.	24
16	An illustration of how the LLC miss files were generated by the CHERI processor simulator.	25
17	An example of how only partial information is known about the state of tags in the table.	26
18	The rubric for assuming the value of an unknown tag.	27
19	The UML diagram detailing the inheritance of the tag controller classes.	30
20	The Morello Project algorithm for reading tags. Pseudo-memory access operations are purple.	35
21	The Morello Project algorithm for writing tags.	36
22	A snapshot of the tag table when running the <i>Efficient Tagged Memory</i> scheme.	40
23	A closer look at the root table.	40
24	A closer look at the leaf table.	41
25	An illustration of the table of tag counters.	42
26	The Phoenix scheme's three-level hierarchical lookup table.	44
27	The physical layout of the hierarchical table in the Phoenix scheme.	44
28	The Phoenix algorithm for reading tags. Tag working set operations are in red.	47
29	The Phoenix algorithm for writing tags. DRAM allocator operations are orange.	49
30	A visualisation of the implementation of the tag working set, and how the replacement policies choose their victims.	50

1 Introduction

1.1 Tagged Memory Architectures

The paper *Efficient Tagged Memory* [1] describes how tagged memory architectures have been explored in academia and employed in industry many times with various different focuses throughout the history of computer design. Through both academic rigour and market testing they have been proven to provide desirable system-wide security properties such as hardware capabilities [2], pointer integrity [3], watchpoints [4], and information-flow tracking [5].

There are however associated costs that render widespread adoption of tagged memory architectures currently infeasible. With a simplistic scheme, the number of memory accesses that workloads must perform doubles due to the CPU now having to access the tag of the value it writes, as well as the value itself, which brings an unacceptably large time overhead. Also, the introduction of the table in which tags are stored comes with a constant memory consumption cost, which is particularly disadvantageous within the server-side compute industry, where every byte of DRAM is precious.

1.2 Question 1 – Existing Mitigations

Efforts have been and are currently being made to mitigate the overheads of tagged memory. Tag caches are often used in practice to hold tag data closer to the CPU for faster access times. The main area in which mitigation of the memory access latency overhead is found is in increasing the ‘cacheability’ of the tag table in DRAM. That is, by increasing the amount of information about the tags that we store in the fast-access tag cache, we reduce the average latency overhead that is introduced by tagged memory’s new requirement of accessing tag data. An effort of this kind, described in *Efficient Tagged Memory*, has been shown to reduce the common-case DRAM traffic overhead to “well below 1%”.

The first question that my dissertation concerns itself with is as follows:

“Question 1: How do existing implementations aim to mitigate the overheads that tagged memory introduces, and how effective are they at doing so?”

1.3 Question 2 – Potential Areas for Improvement

There has been little documented research on attempts to address the memory consumption cost that renders tagged memory a non-starter in server applications. A potential solution to this problem might lie in the use of a hypervisor to dynamically allocate DRAM to guest OSes. Suppose that the tagged memory system is one which runs a hypervisor – a common setup for servers. Suppose further that the hypervisor has a dynamic DRAM allocator that could offer free physical address regions to its guest operating systems as and when they require it. A novel system involving a dynamically-resizing tag table that can be compressed and decompressed at runtime would allow the guest OSes to reserve and free regions of DRAM that they use for storing their tags as they execute, meaning that each OS will consume large amounts of extra space only when necessary, as opposed to all of them reserving at boot-time the maximum that they might require during any part of their execution. By increasing the ‘compressibility’ of the tag table, we can keep the size of the tag table in each guest OS instance closer to a theoretical minimum, reducing the additional memory consumed by the table.

The second question that my dissertation concerns itself with is as follows:

“Question 2: How can we further reduce the overheads that tagged memory introduces?”

1.4 My Project

The utility of tagged memory architectures motivates the exploration and investigation of existing tag controller schemes in search for opportunities to improve upon them – to increase the ‘cacheability’ and ‘compressibility’ of the tag table. Previously, frameworks for implementing tag controllers have been developed, but they were either proprietary, heavily undocumented, or not intended for public use. However, it is from research performed using these frameworks where the majority of innovation in tagged memory architecture implementations has arisen.

For my project, I aim to develop a framework in which an architect can implement tag controllers of their own design, and evaluate them quantitatively. This would allow rapid iteration of designs, and thus would help to foster innovation. I will use the framework to evaluate a handful of existing tag controller implementations, addressing Question 1. I also aim to develop a tag controller of my own that builds on ideas from existing implementations, proving the utility of the framework both as a tool for understanding the strengths of current controllers and for evaluating new ones. By developing both the framework and my novel tag controller, I hope to be able to provide a contribution to the field of research in this area, and to participate in answering Question 2.

2 Preparation

This chapter explains the relevant background material required to undertake the project and to understand the dissertation. It also details the major pieces of my project, how they fit together, and what was required to produce a successful implementation.

2.1 Background Material

2.1.1 Tagged Memory Architectures

In a tagged memory architecture N -bit values in memory are notionally ‘extended’ with M -bit tags. Doing so can provide desirable security properties like those mentioned in the introduction. *Efficient Tagged Memory* [1] describes how a one-bit tag is sufficient to implement all of the security properties that tagged memory architectures can afford. The extension of values with tags might be implemented in one of several ways, but section III of the paper details how storing all of the tag bits in a single table at the bottom of DRAM is the most practical for single-bit tag (SBT) architectures; this is also the most used technique in practice.

Typically, these architectures mandate allocation of a static region of DRAM at boot-time to act as the tag table. This results in a constant physical memory consumption cost that is proportional to the size of the (taggable) address space.

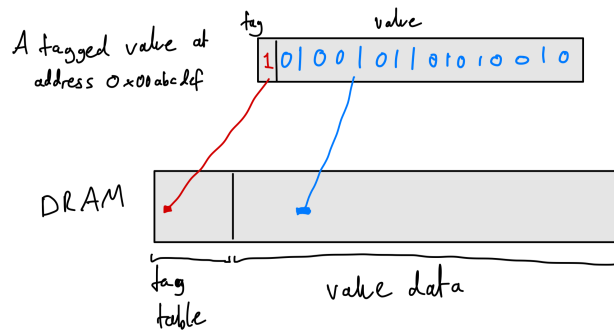


Figure 1: How the two parts of a tagged value are stored in physical memory.

To implement such a scheme in hardware, we use a tag controller. This unit is responsible for intercepting memory access requests and replacing them with appropriate requests that will result in the retrieval of both the value and its tag.

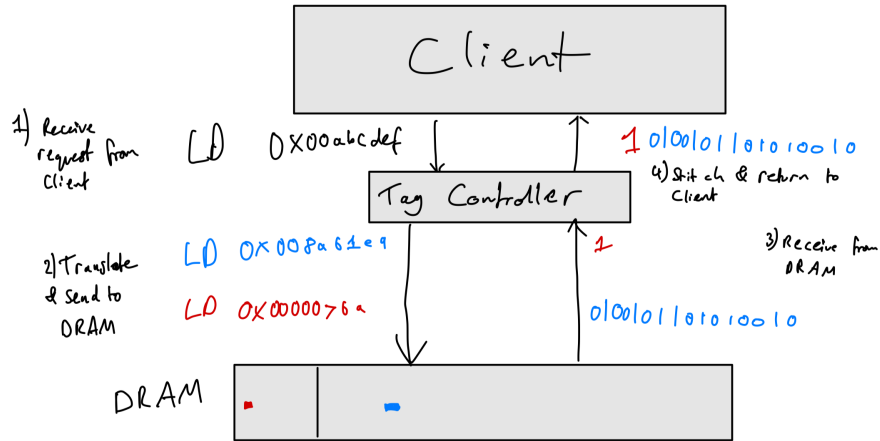


Figure 2: Step-by-step memory access translation.

My project focuses on merged-cache hierarchy architectures. In these, values in the cache are stored unified with their tag – that is, the data caches contain tags as well as values. Section IV of *Efficient Tagged Memory* explains how the alternative, split-cache hierarchy architectures, are ‘unnecessary’, and result in a less efficient and less practical implementation. With merged-cache hierarchies, the tag controller only needs to intercept last-level cache (LLC) misses. The tag controller therefore interfaces with the LLC and DRAM.

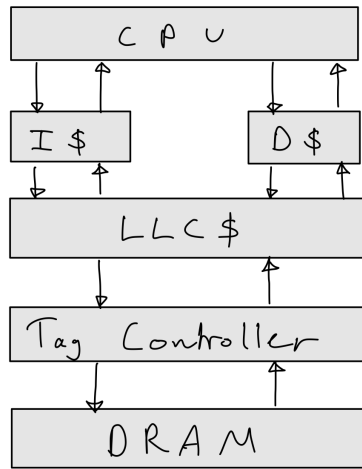


Figure 3: How the parts of the memory system interface.

2.1.2 CHERI

My project focuses on the CHERI architecture model. The model defines a single-bit tagged memory architecture, where 64-bit pointers can be replaced with 128-bit ‘capabilities’ that are also given a 1-bit tag, which allows hardware-enforced security invariants to be guaranteed about these special values, and thus also about executions of programs that use them. There is more information about the CHERI model on the website for the Department of Computer Science and Technology at the University of Cambridge [6].

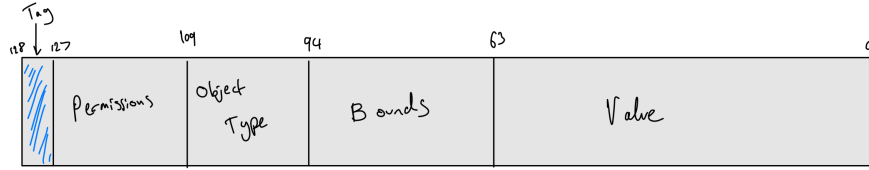


Figure 4: A CHERI capability.

My project focuses particularly on the CHERI architecture as it is a speciality of the university and of my supervisor, and it is also the tagged memory architecture that I am most familiar with, as it was briefly introduced to us in IB, and I also spent some time working with it over the previous summer during my internship as a member of Arm’s Morello Kernel Team. Much of my work and findings however will be transferable to other tagged memory architectures.

2.1.3 A Basic Scheme

The most basic scheme has a single flat table at the bottom* of DRAM that contains all of the tags, and nothing more. This table contains one tag bit per 128 bits of regular memory (as a CHERI capability is 128-bits plus a 1-bit tag). The table therefore consumes the bottom 129th of DRAM. The layout of DRAM is as depicted in figure 1.

* Note that some implementations instead put the table at the top of DRAM. This is functionally equivalent, though of course the addresses used will differ.

When the tag controller retrieves tags by reading from the tag table, it uses the following algorithm:

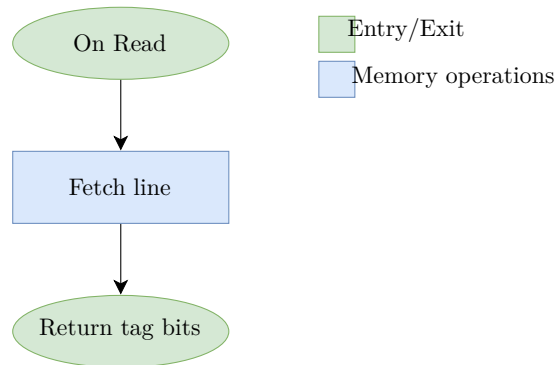


Figure 5: The basic algorithm for reading tags. Memory access operations are blue.

When the tag controller updates the state of tags by writing to the tag table, it uses the following algorithm:

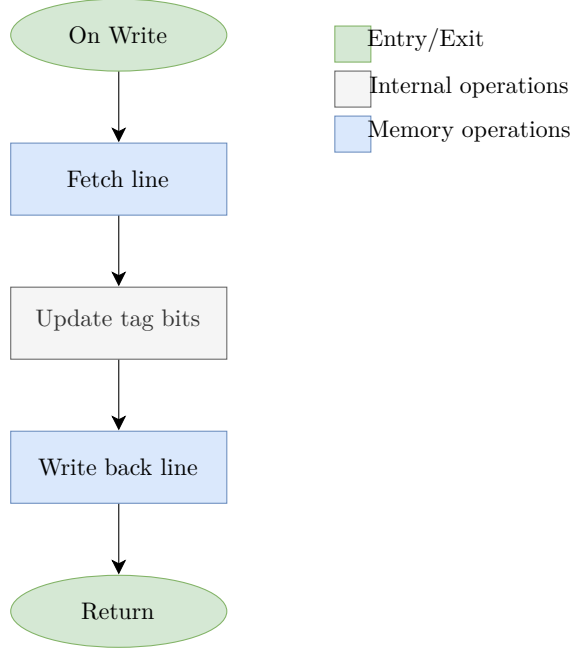


Figure 6: The basic algorithm for writing tags. Operations internal to the tag controller are grey.

As far as the processor and the cache systems are concerned, regular memory still begins at physical address 0x0. The fact that the controller is mandating that the base of the *tag table* is at 0x0, rather than the base of physical memory being here, is abstracted away by the controller’s interface. For clarity, I will refer to the addresses that the processor and caches see as ‘logical physical addresses’, and those that the tag controller emits and DRAM sees as ‘actual physical addresses’. These are both distinct from virtual addresses, which are not relevant here.

Also, note that everything here is done in terms of cachelines, as the tag controller’s client is a cache – the tag controller receives logical base addresses of cachelines from the LLC, rather than those of individual values, and it must return entire cachelines to the LLC.

So, suppose the last-level cache misses the cacheline with logical base address a . It sends a request to the tag controller to retrieve the cacheline that lives at logical address a , and also its corresponding tags. The controller then uses the following formulae to calculate which actual physical addresses it needs to access in DRAM:

$$\text{Actual Value Cacheline Address} = a + \frac{\text{Physical Memory Size}}{128} \quad (1)$$

$$\text{Actual Tag Cacheline Address} = \left\lfloor \frac{a}{128} \right\rfloor_{64} \quad (2)$$

Where the notation $\lfloor x \rfloor_y$ denotes the nearest multiple of y that is less than or equal to x – i.e. rounding x down to the nearest multiple of y .

For the actual base address of the value cacheline, we must take into account the offset of the value portion of DRAM. The value portion begins immediately after the end of the tag table. The tag table has size $\frac{\text{Physical Memory Size}}{128}$, thus we add this value onto the logical address of the line, a , to get the actual physical address of the line.

The actual base address of the tag cacheline will index into the tag table. In this scheme, the tags are arranged

in the same order as the 128-bit values are themselves in DRAM. That is, the first bit of the tag table is the tag of the first 128-bit value in DRAM. The second bit is that of the second 128-bit value, and so on. As we assume each cacheline is the industry standard of 64 bytes, and that each tag corresponds to 128 bits (16 bytes), there are 4 tags per value cacheline. Thus, **one cacheline of tags from the tag table** holds the tags for **128 value cachelines** (see equation 3) By dividing a by 128 we get the location in the tag table that corresponds to the location of the value in the value portion of DRAM. We then must calculate the base address of the tag cacheline which this location lies in, which is done by rounding down to the nearest multiple of 64, as the tag cachelines are 64 bytes.

$$\frac{512 \text{ tags per tag cacheline}}{4 \text{ tags per value cacheline}} = 128 \text{ value cachelines covered by one tag cacheline} \quad (3)$$

2.1.4 The Importance of a Tag Cache

On retrieving a cacheline, the tag controller must make two memory accesses; one to retrieve the value cacheline, and another to retrieve the tag cacheline, from which the appropriate tags are extracted and merged with the values, before returning the result to its LLC client. The fact that we now have to make several memory accesses when retrieving a single cacheline is why we observe an overhead in lookup latency. Typically, the access to the value cacheline and the access to the tag cacheline can be pipelined, but this still increases the latency by a factor of about 1.1. On the other hand, the algorithm for writing contains two accesses with a data dependency, and thus these cannot be pipelined, so the latency of LLC write misses increases by a factor of 2.

Without adding another level of data cache, we cannot avoid accessing the value cacheline in DRAM. What we can do though is introduce a new cache at this level of the memory hierarchy, just below the LLC and alongside the tag controller, that stores cachelines from *the DRAM tag table*. This is known as the tag cache.

When accessing memory with the presence of a tag cache, we first check to see if the tag cacheline we want is in the tag cache. This request can be performed in parallel to the DRAM lookup of the value cacheline. If the tag cache happens to hit, then we avoid the second DRAM access entirely, and eliminate the overhead. Naturally, this motivates efforts to maximise the hit rate of the tag cache.

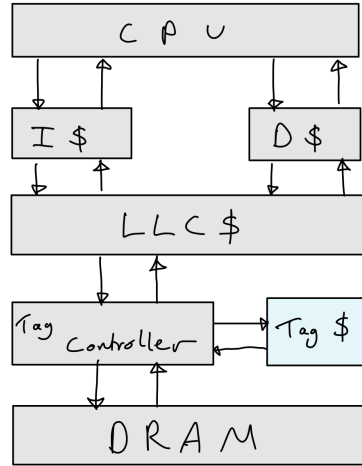


Figure 7: Where the tag cache sits in the memory system.

2.1.5 *Efficient Tagged Memory's Hierarchical Scheme*

One of the tag controller schemes that my project focuses on is the scheme described in *Efficient Tagged Memory*. This is in part because of its simplicity, but also because my supervisor contributed to the paper; his familiarity with the scheme allowed him to guide me easily and effectively where necessary.

One way to increase the hit rate of the tag cache is to store more information about the tag table within it. This can be done by simply increasing its size. However, there is an alternative – by adjusting the scheme, we can increase the ‘cacheability’ of the tag table. That is, we can fit more information about the tag table in the same size tag cache. Section VI of *Efficient Tagged Memory* details a tag table scheme that does just this.

This scheme uses a two-tiered hierarchical tag table. The ‘leaf’ table is identical to the table described in the basic scheme, with one tag bit per 128-bit value. The ‘root’ table contains bits that hold information about *the state of the tags in the leaf table*.

One bit in the root table corresponds to 512 bits in the leaf table – an entire cacheline of tags. The root bit is a 0 if all 512 bits in the corresponding leaf cacheline are 0, otherwise it is a 1.

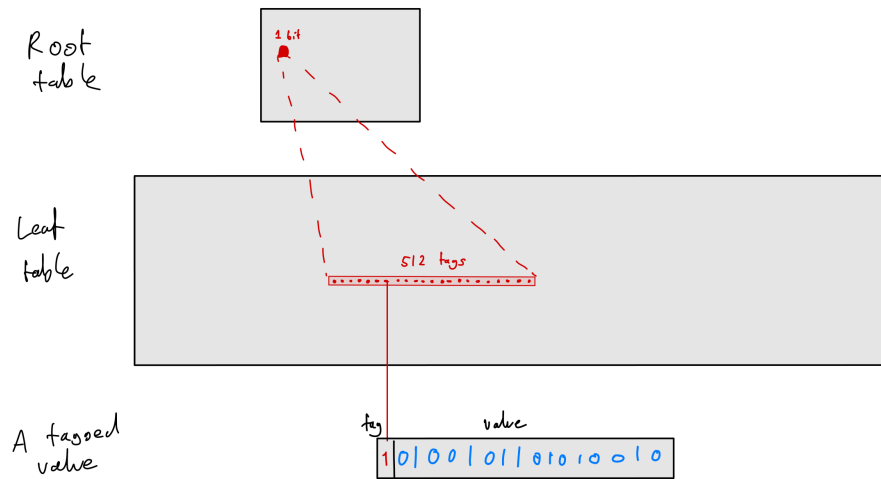


Figure 8: *Efficient Tagged Memory's two-level hierarchical lookup table.*

The tag controller has a new algorithm for both reading and writing the state of tags in the table. The algorithm for reading is as follows:

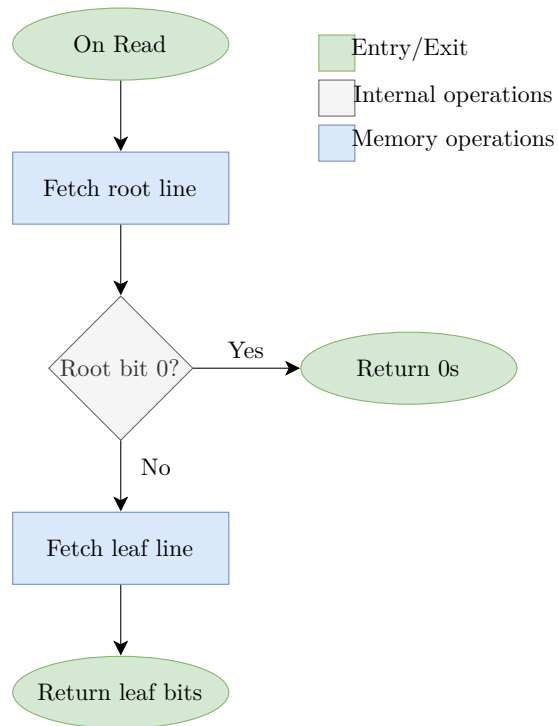


Figure 9: The ETM algorithm for reading tags.

The algorithm for writing is as follows:

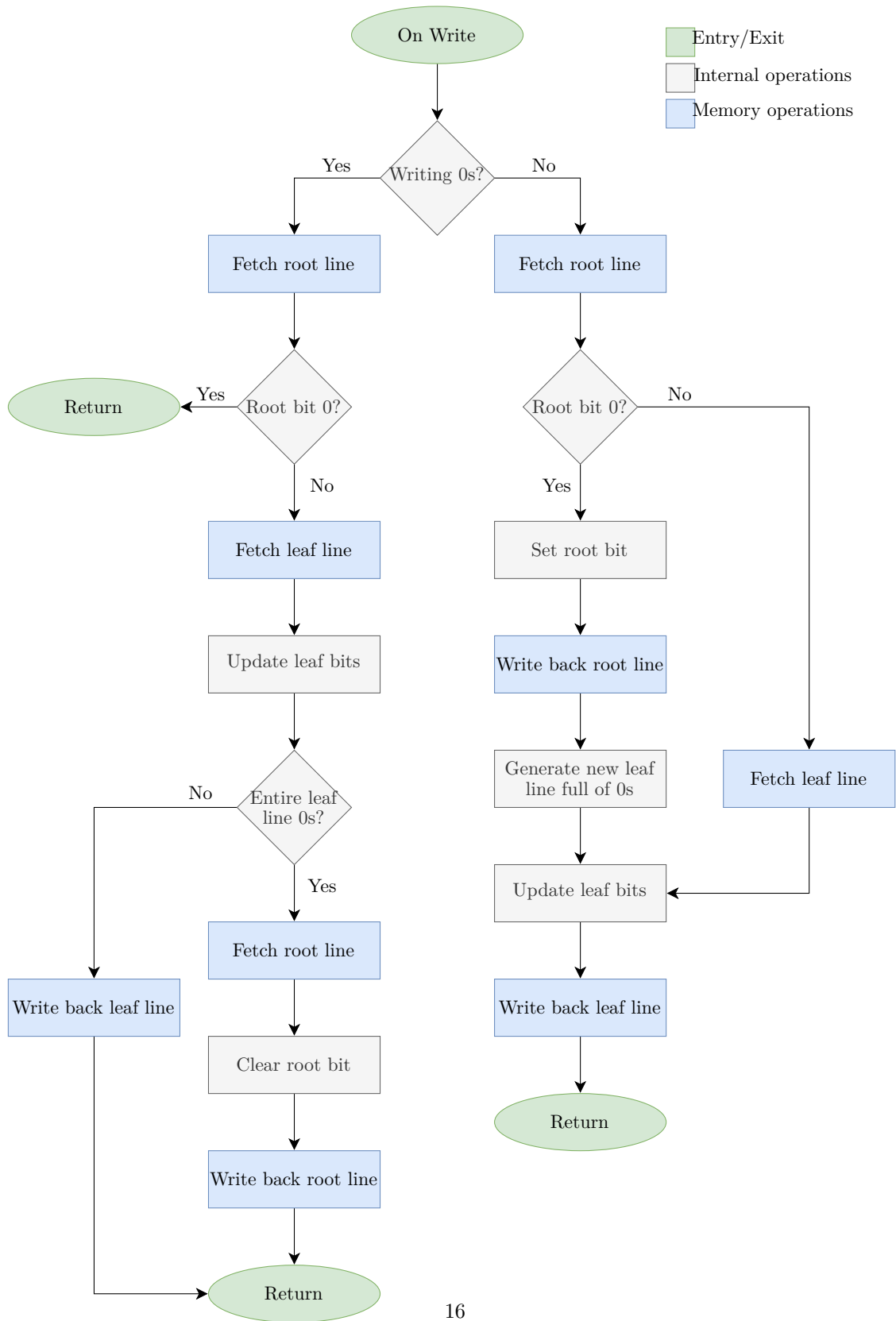


Figure 10: The ETM algorithm for writing tags.

The tag cache may hold lines from either the root table or the leaf table. For each *cleared bit from the root table* that resides in the tag cache, we are essentially containing information about the state of an entire leaf cacheline whilst only consuming one bit of cache space. When executing typical CHERI workloads, it is common to find cleared root bits. It is even common to find entire cachelines of cleared root bits. This is due to the nature of these workloads – the capabilities tend to be focused into small regions of memory, rather than spread out across the entire address space.

The tag cache is even more important here, as these algorithms contain more pairs of operations that have data or control dependencies, meaning that there are proportionally less opportunities to pipeline DRAM accesses, despite potentially having to make more of them in total.

Thankfully, though, this scheme observes a very high hit rate for the tag cache. Thus, whilst in some cases we triple the number of memory accesses for a given LLC miss, the paper finds that there is only a very small overall increase in DRAM traffic overhead, due to the tag cache being able to hold plenty of information about the tags, and it therefore being scarcely necessary to perform additional DRAM accesses. Part of my project is to reproduce these results.

In DRAM, the root table lives below the leaf table. The leaf table lives below the value portion of DRAM (see figure 11).

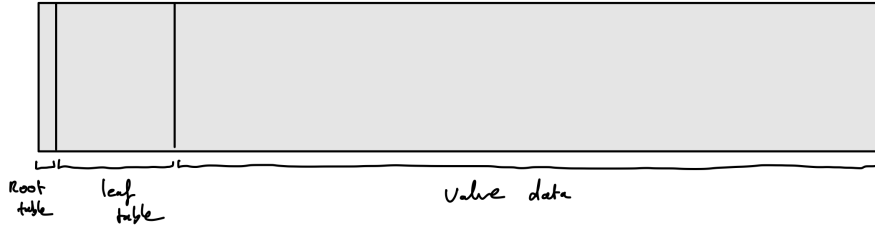


Figure 11: The physical layout of the hierarchical table.

Suppose again that the last-level cache misses the cacheline with logical base address a . The controller uses the following formulae to calculate which actual physical addresses it might need to access in DRAM:

$$\text{Actual Value Cacheline Address} = a + \frac{\text{Physical Memory Size}}{65536} + \frac{\text{Physical Memory Size}}{128} \quad (4)$$

$$\text{Actual Leaf Tag Cacheline Address} = \left\lfloor \frac{a}{128} \right\rfloor_{64} + \frac{\text{Physical Memory Size}}{65536} \quad (5)$$

$$\text{Actual Root Tag Cacheline Address} = \left\lfloor \frac{a}{65536} \right\rfloor_{64} \quad (6)$$

The formulae for the actual base addresses of the value cacheline and the leaf tag cacheline are as they were in the simple scheme, with an additional constant of $\frac{\text{Physical Memory Size}}{65536}$ added on. This is to adjust for the offset that the root table adds.

The actual base address of the root tag cacheline will index into the root table. The address is calculated by similar logic to that of the leaf tag, and equation 7 explains the presence of the constant 65536.

$$\begin{aligned} & \frac{512 \text{ leaf tags per leaf cacheline}}{4 \text{ tags per value cacheline}} \times 1 \text{ leaf cacheline per root tag} \times 512 \text{ root tags per root cacheline} \\ & = 65536 \text{ value cachelines covered by one root cacheline} \end{aligned} \quad (7)$$

2.1.6 The Morello Project Scheme

As I mentioned prior, during my internship with Arm over the previous summer I contributed to their Morello project. Morello is a research project that defines a microarchitecture based on the CHERI model. Another tag controller scheme that my project focuses on is the scheme used in the Morello microarchitecture. This is due to my own and my supervisor’s familiarity with CHERI, and the Morello architecture and microarchitecture. There is more information about the Morello project at the project’s website [7].

This scheme aims to do a similar thing to the one described in *Efficient Tagged Memory*, but in a different way. Instead of introducing a new, more information-dense table that can be stored in the existing tag cache, it introduces a new tag cache that holds information-dense data.

This scheme uses a simple flat DRAM tag table, just like the one in the basic scheme, but it employs two tag caches. One is a cache reserved for holding tag cachelines that contain *at least one* set tag, call this the non-zero cache. The other is a cache reserved for holding tag cachelines that contain *no* set tags – i.e. lines that are fully cleared, call this the zero cache. The former cache is just like a regular cache; for each cacheline, there is an entry in the cache that stores both the data at the address and the *address-tag** of the cacheline’s base address. However, the latter cache only needs to store the address-tag of the cacheline’s base address, as it must be that the entire cacheline contains zeroes, given that it is in the zero cache, thus there is no need to store any actual data. Requests can be issued to and handled by the two caches in parallel, further mitigating overhead.

* The *tag of a cacheline’s base address* is defined as the upper K bits of the base address of the cacheline (for some K defined by various system parameters). This is the academic-standard name for this value, but it unfortunately clashes with the tagged memory *tags*. To disambiguate, I use the term *address-tag* hereon to refer to the former, and simply *tag* for the latter.

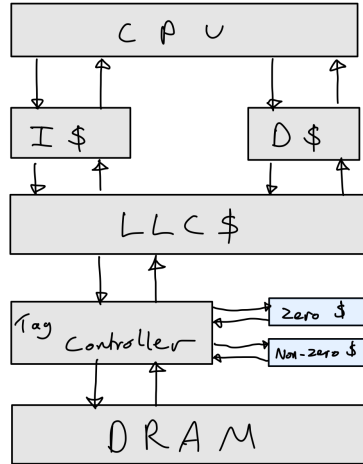


Figure 12: The layout of the two tag caches in the Morello memory system.

As the tag table has not changed from the simple scheme, the formulae used by the controller for translating addresses are the same (equations 1 and 2), and so are the two algorithms (figures 5 and 6).

2.2 Three-Stage Simulation

Part of my project was to develop a simulator that sits in the middle of a three-stage simulation pipeline. The full pipeline enables the simulation and analysis of the tag table and cache behaviour of given CHERI tag

controllers.

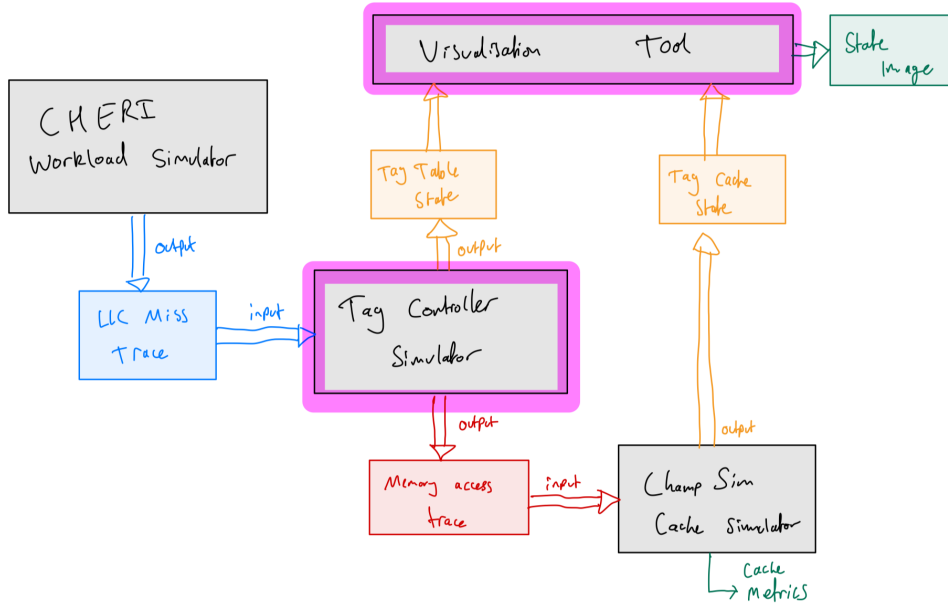


Figure 13: A black-box diagram explaining the three stage simulation.

In this subsection, I explain each of the three stages of simulation.

2.2.1 Workload Execution – LLC Miss Simulation

A former student had previously created a simulator that consumes a program binary and simulates the execution of the binary on a CHERI architecture. The simulator outputs trace files that log the last-level cache misses that occurred during the simulated execution.

I have been given access to a collection of trace files by my supervisor. These files were logged whilst running various benchmark binaries. The simulator I developed takes these trace files as input. As the tag controller only receives information about LLC misses, the trace files contain sufficient information to determine which accesses would be made by the simulated tag controller algorithm if it were to run the given workload.

2.2.2 A Tag Controller Simulator

The main objective of my project was to develop a tag controller simulator in C++. When given a workload it should simulate the memory accesses that a specified tag controller algorithm would result in. The idea is that this simulator can be leveraged to provide insight into the performance and inner workings of the tag controller algorithm being simulated, providing useful information to an architect.

2.2.3 ChampSim – An Appropriate Cache Simulator

ChampSim is an open-source trace based cache simulator written in C++. There is more information about ChampSim at the ChampSim GitHub profile page [8].

My simulator interfaces with ChampSim, having it handle the simulation of the tag cache(s). My simulator provides ChampSim with a trace file of memory accesses that the simulated tag controller makes during its execution of the workload. ChampSim then provides the user with metrics regarding cache access frequency and hit ratio. The user can specify parameters of the tag cache such as cache size and number of ways. ChampSim is a good choice of cache simulator because its open-source C++ code allowed me to easily extend it to add necessary instrumentation. Also, it has built-in cache prefetchers which can be enabled or disabled simply by adjusting a configuration file. This lets an architect see how much difference adding a prefetcher to their tag cache would make to performance.

I opted to use an existing cache simulator instead of developing my own because the focus of this project is on investigating tag controllers, as opposed to caches in general, and my supervisor informed me that there are likely many opportunities for bugs to arise when developing a cache simulator of my own, and thus it is a more appropriate use of my time to focus more on tag controller research rather than debugging software that has already been written.

2.2.4 Visualisation Tool

It would be useful for an architect to be able to somehow visualise how the state of the tag table and the tag cache changes as the workload executes. Having this insight would inform new approaches and justify design choices for a novel tag cache.

As part of my project, I developed a tool that allows a user to take ‘snapshots’ of the tag cache at chosen points of workload execution, and view them afterward. The images depict the state of the tag table and the tag cache at that point of execution.

2.2.5 Requirements Analysis

There are several requirements that a successful simulator must meet:

- It must be able to decode the LLC miss input trace file format.
- It must be able to correctly simulate the sequence of memory accesses that the simulated tag controller algorithm would perform when executing the given workload.
- It must be able to produce an accurate output trace file of these accesses that is decodable by ChampSim.

On top of this, for the visualisation tool to be functional and useful, the following are required:

- My simulator must be able to output the state of all tags at given points of execution.
- The system must be able to extract the state of the tag cache from ChampSim at given points of execution.
- The visualisation tool must be able to read in the captured states of the tag table and the tag cache and produce an image that accurately depicts this state.

2.3 Starting Point

The following bullet points detail relevant work completed and preparations made before beginning the project:

- I had read the paper *Efficient Tagged Memory*.
- As I have mentioned, over the summer of 2023 I interned at Arm in the Morello Kernel Team, a team working to port the Linux kernel to Arm’s Morello architecture, which is based on the CHERI model. This gave me a chance to get more familiar with the CHERI model.

- My project uses ChampSim, the open-source cache simulator. I also extend this program by adding logging support, by branching from commit `#2b8d3fc28abb6072d7675228418fa7cfe862d4dc`. I took no part in contributing to any versions of ChampSim prior to this commit.

3 Implementation

This chapter describes the work I did for the project. It outlines the structure of the programs I developed, and goes in-depth into how exactly I implemented the described functionality. It also details the process by which I designed my novel tag controller scheme.

3.1 Simulator

The simulator consists of 3 main parts: the trace decoder, the tag controller, and the simulated memory. Figure 14 shows how these parts fit into the project's pipeline of information flow.

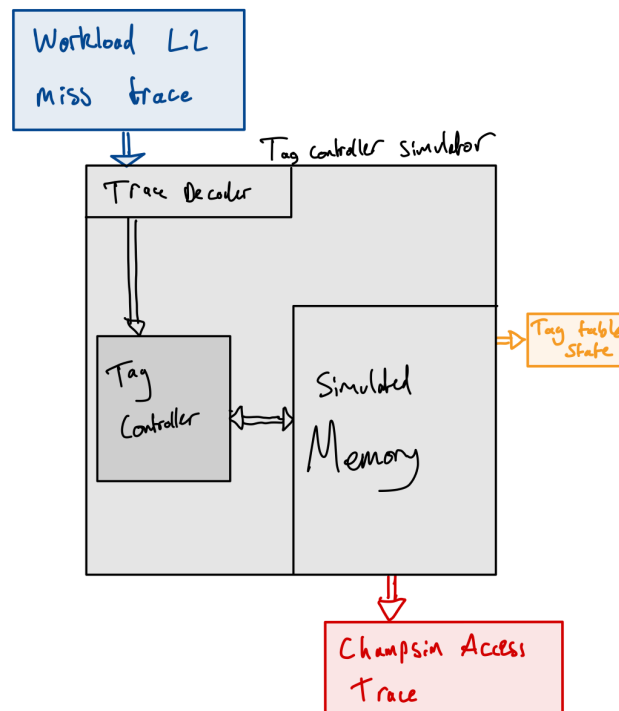


Figure 14: An explanation of how the three main parts of the tag controller simulator interact with the inputs and outputs, and with each other.

3.1.1 Simulator Framework

The simulator is a program with a command line interface defined in `main.cpp`. It allows a user to specify the following:

- the name of the tag controller scheme to simulate
- the path of the LLC miss input trace file
- the path of the tag table initial state input file
- the path of the output trace file
- the maximum number of LLC misses to process (optional, default 5 million)

- the path of the output log file to log the tag table state into (optional)
- the indices of instructions after which a tag table state log entry should be made (known as ‘log points’; optional)

`main.cpp` then parses these arguments, and opens the files specified. It creates an instance of a `Decoder` and has it decode the input files. It then creates an instance of a `Controller` of the specified scheme, and a `Simulator`. It then has the `Simulator` simulate the tag controller’s execution when running under the workload of the input trace files.

The `Simulator`’s job is simply to iterate through the decoded LLC misses, passing each to the controller’s `handleRead` or `handleWrite` method, for reads and writes respectively. This emulates the LLC streaming its sequence of misses to the tag controller in a hardware implementation.

Code Snippet 1 `Simulator::processTrace`

```

1  static size_t processTrace(Decoder &decoder, Controller &controller, size_t n,
   ↪ unordered_set<size_t> &log_points) {
2      size_t processed = 0;
3
4      vector<memAccess> buffer(CHUNK_SIZE);
5      for (size_t at = 0; at < n; at += CHUNK_SIZE) {
6          size_t read = decoder.readLLCMisses(buffer, min(CHUNK_SIZE, n-at));
7
8          int i = 0;
9          for (size_t remaining = read; remaining > 0; --remaining) {
10             if (buffer[i].type == ACCESS_TYPE_READ)
11                 controller.handleRead(buffer[i]);
12             else controller.handleWrite(buffer[i]);
13
14             if (log_points.find(processed) != log_points.end()) {
15                 cout << "LOG @ " << processed << endl;
16                 controller.dump_table();
17             }
18             ++i;
19             ++processed;
20         }
21     }
22
23     return processed;
24 }
```

At the end, `main.cpp` reports the total number of LLC misses that were handled, and the total number of memory accesses the tag controller made. It also saves and closes the output file(s).

3.1.2 Trace Decoder

The `Decoder`’s job is to retrieve information from the input trace files and provide it to whichever calling function requested it.

On its construction, it is given the file objects for the two input trace files. It has two public methods, `readLLCMisses` and `getInitialTags`.

`readLLCMisses` is what the `Simulator` uses to decode LLC misses from the LLC miss input trace file. The file contains a sequence of structs with no padding between them. The structs have fields which begin at specific byte indexes, as detailed in figure 15.

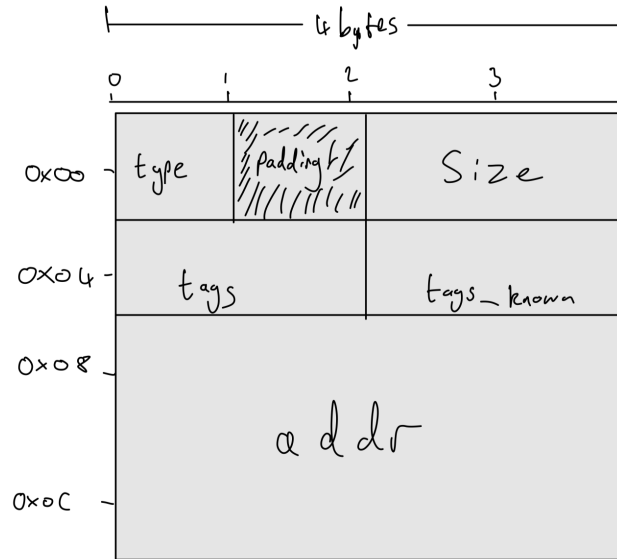


Figure 15: The layout of an `llcMiss` struct.

By using the correct format, the decoder reads out the fields of each LLC miss entry from the file, one at a time. For each entry, it constructs an `llcMiss` struct, which is defined in `trace.h`.

The decoder uses `zlib` to read the compressed version of the trace files, rather than requiring the user to first decompress them. This saves them the pains of having to wait for the entire file to be decompressed rather than just the bytes that they are interested in, and of having to store decompressed files on disk whose filesize can be on the order of terabytes.

The final step of this process is to *furnish* the decoded `llcMiss` entry to convert it to a `memAccess` struct (also defined in `trace.h`) before writing that into the buffer. *Furnishing* refers to a process of filling in partially-unknown information about the tags as best we can given the information we do have.

To explain this, we must first understand some background about the source of the input trace files. The `CHERI` simulator that generated the input trace files worked like so:

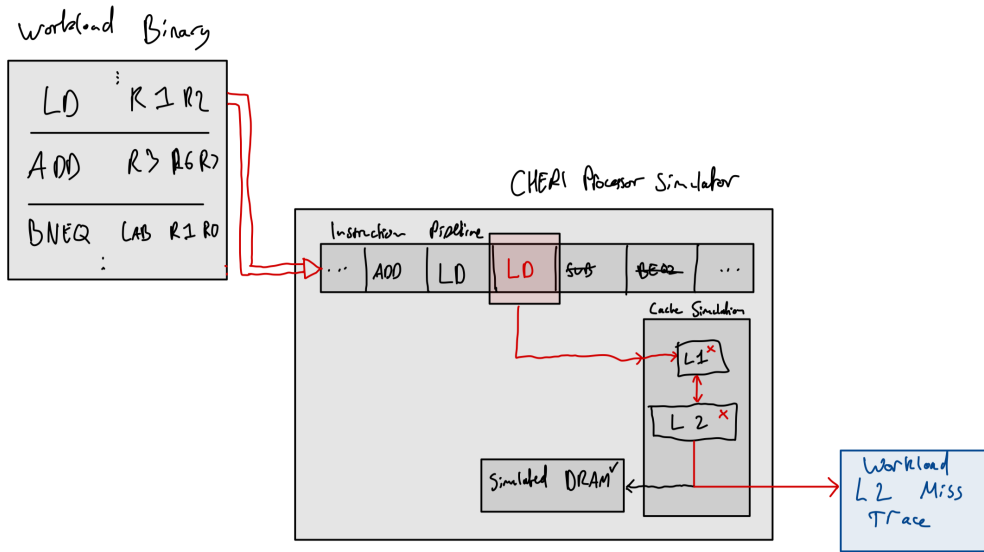


Figure 16: An illustration of how the LLC miss files were generated by the CHERI processor simulator.

Instructions were decoded from the program binary, and issued to the simulated CPU. When processing memory access instructions that resulted in LLC misses, an entry was made to the LLC miss trace file, detailing the miss. The entry attempts to describe the state of the four tag bits that were read (in the case of a read), or the state that we update the four tag bits to have (in the case of a write). Figure 16 describes this process.

However, the information about the state of *all* four tag bits is not necessarily known at the time the entry was logged. At this time, the simulator only has information about the tag that was read/written by the instruction that caused the miss, and information about the other tags that could be inferred from previous memory access instructions. Figure 17 details an example of this. Initially, we know nothing about the state of the tags. Suppose that an instruction sets tag 2 of a cacheline to be high. Suppose further that a later instruction reads tag 3 of this same cacheline; we can know the value of tag 3 from the instruction's result, and we also know that tag 2 is still high, assuming no other instruction has caused it to change since the previous instruction set it. In the case of the other two tags in the cacheline, the simulator does not know what value they have at this time.

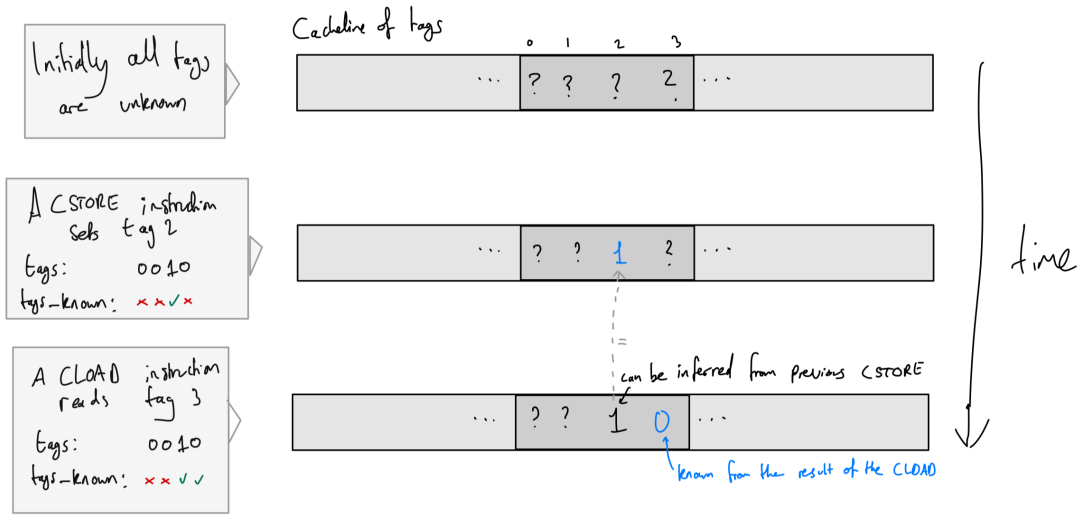


Figure 17: An example of how only partial information is known about the state of tags in the table.

This lack of knowledge about particular tags is embedded in each entry in the LLC miss trace file, and upon being decoded, it is stored into the `tags_known` field of the resulting `llcMiss` struct. This field is a bitmask indicating for which of the four tags we have definite information, and for which we do not.

To attempt to gain information for tags that we do not know about, we can refer to the initial state file that is associated with the LLC miss trace. This file contains information regarding what can be known about the *initial state of the tags* in memory. In this file, there is an entry for each tag in the DRAM tag table. The `Decoder`'s private method `getInitialTag` is responsible for making an educated guess at the value of a tag of interest based on the initial state file's entry for that tag.

The `getInitialTag` method first reads and decodes the entry for the tag of interest, obtaining two fields: a boolean named `tag` and a value of the enumerated type `initialAccessType`, which is defined in `trace.h`. The following table details how the `assumeTag` function generates a best-guess initial tag value from these two fields:

Initial access type	Tag	Explanation	Assumed tag
INITIAL_ACCESS_TYPE_CLOAD	0	At some point, before the tag was modified, a capability-load instruction read this tag's value as a 0, so the tag was definitely initially 0.	0
INITIAL_ACCESS_TYPE_CLOAD	1	At some point, before the tag was modified, a capability-load instruction read this tag's value as a 1, so the tag was definitely initially 1.	1
INITIAL_ACCESS_TYPE_CSTORE	0	A capability-store instruction overwrote this tag's value to 0 before the initial value was discovered. We assume the capability-store did not change the value.	0
INITIAL_ACCESS_TYPE_CSTORE	1	A capability-store instruction overwrote this tag's value to 1 before the initial value was discovered. We assume the capability-store did not change the value.	1
INITIAL_ACCESS_TYPE_LOAD	0/1	No capability instructions accessed this tag so we have no concrete information, but it is likely that the value associated with this tag is non-capability data, and thus has its tag cleared.	0
INITIAL_ACCESS_TYPE_STORE	0/1	No capability instructions accessed this tag so we have no concrete information, but it is likely that the value associated with this tag is non-capability data, and thus has its tag cleared.	0

Figure 18: The rubric for assuming the value of an unknown tag.

Putting it all together, the `furnish` method converts decoded `llcMiss` structs into `memAccess` structs by using the `getInitialTag` method to fill in gaps. The `getInitialTag` method in turn uses the above assumption scheme defined by `assumeTag` to make an educated guess at what the tag previously was, though at times it must make assumptions due to lack of information.

The `Decoder` class also has a `getInitialTags` method that produces an entire cacheline's worth of initial values of tags in the tag table. It uses the same methods, and the same assumptions for making educated guesses, as the `getInitialTag` method does. It is used by `Controllers` when they initialise the state of their `Memory`. This method is more efficient than simply calling `getInitialTag` 512 times as it performs a single disk read to retrieve all 512 relevant entries from the initial state file at once. It also provides a more useful interface to the calling methods, as they operate on cacheline-granularity rather than bit-granularity.

3.1.3 Simulated Memory

`Memory.h` defines the `Memory` class, which represents the tag table stored in memory. This class is responsible for the following:

- Providing an interface to the tag controller, allowing it to read from and write to the state of the tag table
- Keeping track of the state of the tag table as the simulation progresses

- Logging all memory accesses that the tag controller makes

The class does **not** keep track of value data – it only records tag data. This simulation is not concerned with values; only tags.

The `Memory` keeps track of the state of each cacheline in the tag table by using a `std::map` that maps simulated physical addresses to `Cachelines`. A `std::map` is an appropriate data structure for two reasons: 1) it has $O(1)$ lookup time, which is key for providing fast simulation runtimes due to the frequency of tag table accesses and the large size of the table, and 2) it maintains a total ordering of its entries over their keys, meaning that when we dump the state of the tag table to the output log, we can guarantee that we traverse the bytes in order of address. An array would also provide these properties, but in cases where we might want to divide the table into chunks that are spread across the entire physical address space, an array would either need to reserve space that spans this entire range, or have a programmer-defined translation scheme for translating addresses into array indices. Reserving space for the entire physical address space is wasteful as only a very small fraction of it would be used, and a programmer-defined translation scheme adds unnecessary complications for the programmer. On the other hand, a map only needs to have as many entries as the number of cachelines that the tag table can store, and no extra translation scheme needs to be defined.

A `Cacheline` is a C++ alias for the type `std::array<uint8_t, 64>`, representing the 64 bytes that a real cacheline has.

The `Memory` provides a `doRead` method and a `doWrite` method to the tag controller, allowing it to read and write cachelines to the simulated tag table. It is important to remember that this simulation is not concerned with whether the tag cache would hit or miss when accessing memory. The purpose of this simulation is to determine the *sequence of memory addresses that the tag controller attempts to access* when running under the chosen workload, which is independent of whether or not the cache hits or misses, thus the control flow of the tag controller’s algorithm will always be accurate to a real run, and therefore the output trace produced will still be an accurate trace of the sequence of memory address accessed by a real controller.

After this simulator produces the access trace, we can feed it into `ChampSim`. `ChampSim` then accurately simulates the physical cache, simulating which lines would hit, which would miss, which would be evicted, and so on, all the while recording statistics such as access count and hit count.

Each time the tag controller needs to read a tag cacheline from memory, we want to create an entry into the output trace reflecting this. As well as allowing the tag controller to interact with the state of the tag table, the `doRead` and `doWrite` methods also take the responsibility of writing the addresses accessed to the output trace. They do this by calling the private `traceRead` and `traceWrite` methods, which simply append a populated `champsimInstr` struct to the file, building up a trace file that `ChampSim` can parse.

There are also cases where we want to read from or update the state of the tag table without writing an entry to the output trace file, which I mention later. The `peek` and `set` methods allow this ‘silent’ interaction with the tag table.

The `Memory` class has two getter methods – `getNumAccesses` and `getPrevLogged`. The former returns the number of entries made into the output trace file. The latter returns a vector of all entries that have been made – this is used when testing.

The class also has a method `dump_table`, which writes each bit of the current state of the tag table into the tag table state output log file.

3.1.4 Tag Controller

Code Snippet 2 Controller class

```
1  class Controller {
2
3  protected:
4      Memory memory;
5
6  public:
7      explicit Controller(gzFile output_trace, ofstream &output_log) : memory(output_trace,
↵  output_log) {}
8
9      virtual void setup(Decoder &decoder) = 0;
10
11     virtual uint16_t handleRead(memAccess ax) = 0;
12
13     virtual void handleWrite(memAccess ax) = 0;
14
15     void dump_table() {memory.dump_table();}
16
17     int getNumAccesses() {return memory.getNumAccesses();}
18
19     vector<champsimInstr> getPrevLogged() {return memory.getPrevLogged();}
20
21     virtual void reportStats() {}
22
23     virtual ~Controller() = default;
24 };
```

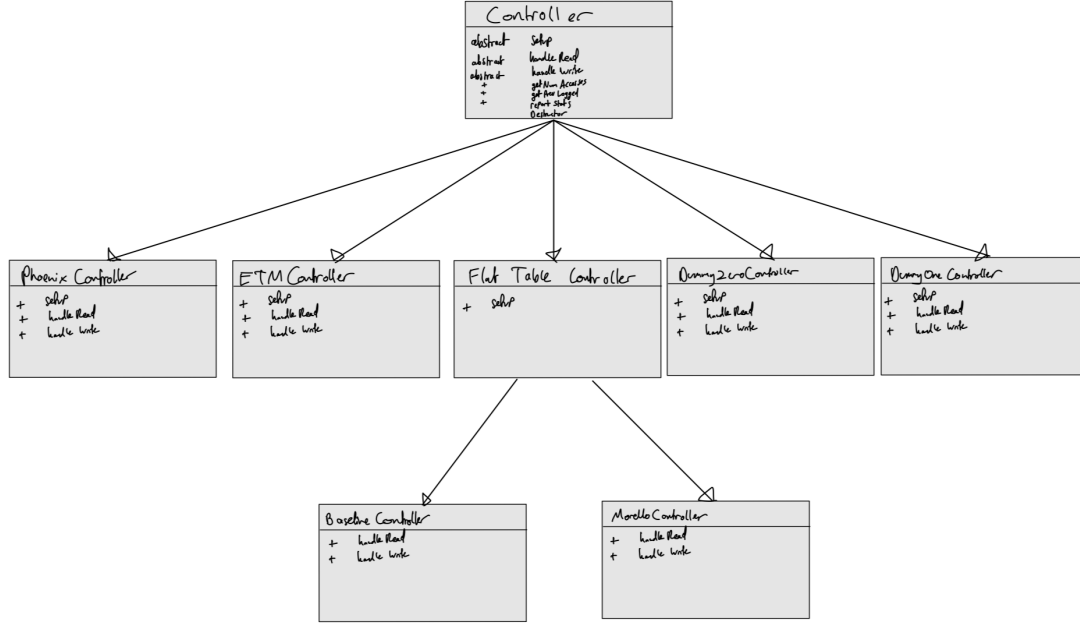


Figure 19: The UML diagram detailing the inheritance of the tag controller classes.

A tag controller is implemented as a concrete class that derives from the abstract **Controller** class. Each tag controller has a **Memory**, to which it sends requests during its operation.

Being abstract, this class also details the methods that a concrete tag controller must implement: **setup**, **handleRead** and **handleWrite**.

The **setup** method is to be called immediately after the **Controller** instance is constructed, so that the state of its **Memory** is correctly initialised. This is done with help from the passed **Decoder**, which reads from the initial tag table state input file. This method is abstract because different tag controllers are part of different schemes, and different schemes have different tag table layouts (the Morello Project’s flat table layout from the *Efficient Tagged Memory*’s hierarchical table layout, for instance), thus the algorithm to initialise them will differ.

The **handleRead** and **handleWrite** methods implement the algorithms that the tag controller follows when handling LLC misses. As this simulator is only concerned with tag data and not value data, a concrete implementation of this method only needs to perform the tag table accesses of the algorithm, and not the value access, or the merging of values with tags.

3.1.4.1 Flat Table Controllers

FlatTableController.h defines an abstract class **FlatTableController** that extends **Controller**. This class partially defines a specific type of tag controller – those that use a flat table, such as the controllers from the basic scheme and the Morello Project scheme. Controllers of this type have two things in common: how logical addresses are translated into actual ones, and how the tag cache should be initialised at the beginning of a simulation.

Code Snippet 3 FlatTableController::translateToTagAddr

```
1 static std::pair<uint64_t, uint16_t> translateToTagAddr(uint64_t addr_base) {  
2     uint64_t tag_base = ((addr_base / 128) >> 6) << 6;  
3     uint16_t tag_offset = ((addr_base >> 6) * 4) % 512;    // index into the tag cacheline  
4     return {tag_base, tag_offset};  
5 }
```

The translation from a logical cacheline base address to the corresponding actual tag cacheline base address is detailed in equation 2. FlatTableController's `translateToTagAddr` method does exactly this (see listing 3). For a flat table controller, the tag cache is initialised by simply iterating across the entire initial tag table state and populating the cache's cachelines.

By implementing a single abstract class from which several other classes can inherit instead of duplicating the functionality across several base classes, we reduce the size of the codebase and increase maintainability due to only having to change a single implementation when changes are necessary. It also makes it easier for an architect to implement their own flat table controller, as they already have the abstract class as a starting point. Additionally, it becomes true that by testing this functionality for a single base class, we test it for all of them simultaneously.

3.1.4.2 Basic Scheme

`BaselineController.h` defines a concrete class `BaselineController` that implements the tag controller from the basic scheme. The class inherits from `FlatTableController`.

The implementations of `handleRead` and `handleWrite` are very simple, and it is clear how the code directly corresponds to the algorithms from figures 5 and 6.

Code Snippet 4 BaselineController class

```
1  class BaselineController : public FlatTableController {
2  public:
3      BaselineController(gzFile output_trace, ofstream &output_log) :
4      ↪ FlatTableController(output_trace, output_log) {}
5
6      uint16_t handleRead(memAccess ax) override {
7          pair<uint64_t, uint16_t> result;
8          uint64_t leaf_base_addr;
9          uint16_t leaf_cacheline_index;
10         Cacheline leaf_line;
11
12         result = translateToTagAddr(ax.addr);
13         leaf_base_addr = result.first;
14         leaf_cacheline_index = result.second;
15         leaf_line = memory.doRead(leaf_base_addr);
16         return getTags(leaf_line, leaf_cacheline_index);
17     }
18
19     void handleWrite(memAccess ax) override {
20         uint64_t leaf_base_addr;
21         uint16_t leaf_cacheline_index;
22         Cacheline leaf_line;
23
24         auto result = translateToTagAddr(ax.addr);
25         leaf_base_addr = result.first;
26         leaf_cacheline_index = result.second;
27
28         leaf_line = memory.doRead(leaf_base_addr);
29         modifyTags(leaf_line, leaf_cacheline_index, ax.tags);
30         memory.doWrite(leaf_base_addr, leaf_line);
31     }
32 };
```

3.1.4.3 *Efficient Tagged Memory's* Hierarchical Scheme

ETMController.h defines a concrete class ETMController that implements the tag controller from the scheme described in *Efficient Tagged Memory*.

This class's `setup` method instantiates the *leaf* table in the same way that a flat table controller instantiates its table. Whilst it does this however, it also populates its *root* table, byte by byte. Each time a cacheline of tags is placed into the leaf table, we keep track of whether or not the cacheline contained any set tags, recording a 1 in the current byte if there were any, and a 0 if not. After processing 8 leaf table cachelines, we push the current byte into the current cacheline. After pushing 64 bytes, we `set` the now-completed cacheline in the root table.

Code Snippet 5 ETMController::setup

```
1 void setup(Decoder &decoder) override {
2     Cacheline root_line, leaf_line;
3     uint8_t root_byte;
4     bool no_leaves_set;
5
6     for (uint64_t rl = 0; rl < ROOT_TABLE_SIZE; rl += TAG_CACHE_LINE_SIZE) {
7         for (int rb = 0; rb < TAG_CACHE_LINE_SIZE; ++rb) {
8             root_byte = 0;
9             for (int rt = 0; rt < 8; ++rt) {
10                 root_byte <<= 1;
11
12                 // focus on this single root tag
13                 decoder.getInitialTags((8*rl+8*rb+rt)*TAG_CACHE_LINE_SIZE*8,
↪ TAG_CACHE_LINE_SIZE*8, leaf_line);
14                 memory.set(TAG_CACHE_LINE_SIZE*(8*rl+8*rb+rt) + ROOT_TABLE_SIZE,
↪ leaf_line);
15
16                 no_leaves_set = isClear(leaf_line);
17                 root_byte |= no_leaves_set ? 0 : 1;
18             }
19             root_line[rb] = root_byte;        // push the root byte into the line
20         }
21         memory.set(rl, root_line);           // insert the root line into the cache
22     }
23 };
```

This class defines two auxiliary functions, `translateToLeafAddr`, and `translateToRootAddr`. Just as `FlatTableController` `translateToTagAddr` implemented the appropriate equation from the Preparation chapter to translate a value cacheline's logical base address to the corresponding tag cacheline's actual base address, `translateToLeafAddr` implements equation 5 to translate a value cacheline's logical base address to the corresponding tag cacheline's actual base address in the *leaf table*. Similarly, `translateToRootAddr` implements equation 6 to translate that logical base address to the actual base address of the corresponding cacheline in the root table instead – that is the base address of the cacheline in the root table that contains the root bit whose scope covers the corresponding leaf table line.

Code Snippet 6 ETMController::translateToLeafAddr and ETMController::translateToRootAddr

```
1  static std::pair<uint64_t, uint16_t> translateToLeafAddr(uint64_t addr_base) {
2      uint64_t leaf_base = (((addr_base / 128) >> 6) << 6) + ROOT_TABLE_SIZE;
3      uint16_t leaf_offset = ((addr_base>>6)*4) % 512;          // index into the tag cacheline
4      return {leaf_base, leaf_offset};
5  }
6
7  static std::pair<uint64_t, uint16_t> translateToRootAddr(uint64_t addr_base) {
8      uint64_t root_base = ((addr_base / 65536) >> 6) << 6;
9      uint16_t root_offset = ((addr_base>>6)*4 / 512) % 512; // index into the tag cacheline
10     return {root_base, root_offset};
11 }
```

The implementations of `handleRead` and `handleWrite` are more complex than that of the basic tag controller, but once again the code directly corresponds to the algorithms from the appropriate figures (9 and 10).

3.1.4.4 The Morello Project Scheme

`MorelloController.h` defines a concrete class `MorelloController` that implements the tag controller from the Morello Project scheme. The class inherits from `FlatTableController`.

The Morello project tag controller implementation is a special case. As mentioned in the Preparation chapter, the Morello Project’s microarchitecture uses two distinct caches. My program handles this by producing the access logs for the two caches in two separate simulations. On invoking the program, the user specifies whether they want to simulate the zero cache, or the non-zero cache. The simulator will then run and produce an output trace file that logs *only the accesses made to the specified cache*. For example, if the user specifies that they want to simulate the non-zero cache, then the output trace produced will detail all accesses made to *non-zero* tag cachelines, and not any to *zero* cachelines.

This way, the simulation framework does not need to be adapted to handle multiple caches at once, and the output log can still be passed straight to ChampSim to simulate the chosen cache.

Now though, this means that the algorithms that we implement must change. In simulation, we want the `Memory` to only log a memory access to the output trace if the line that the controller is accessing is a line that *would be held in the cache we are simulating*. To do this, we need a way for the controller’s algorithm to know the state of the cacheline before it decides whether or not to log the memory access. The `Memory`’s `doRead` would tell the controller the state of the cacheline, but as it does this, it has the side-effect of logging the memory access to the output trace file, which is a problem if the line happens to be one that we do not want to log the access of. That is, with this method, we can only know if we actually want to log the access *after* we have called the method and consequently already logged the access.

The solution is to instead use the `Memory`’s `peek` method. With this method the controller can see the state of the cacheline without logging the access, allowing it to decide whether or not to log the access *after* examining the line’s state, and only doing so if it matches the cache being simulated.

Similarly, on updating a tag, we might not necessarily want to log the write memory access, but it is important that we update the state of the tag in the simulation, otherwise the algorithm might later read stale data and make decisions that are not accurate to a real world run of the controller, where the value would have changed in memory. The `Memory`’s `set` method can be used to update the state of a cacheline without logging a write memory access to the output trace.

Figures 20 and 21 detail the new algorithms we must implement. They contain new ‘pseudo-memory access

operations' that correspond to **peeking** and **setting**. These operations are purely for simulation control flow purposes and do not correspond directly to real operations that a physical tag controller would perform.

The algorithm for reading is as follows:

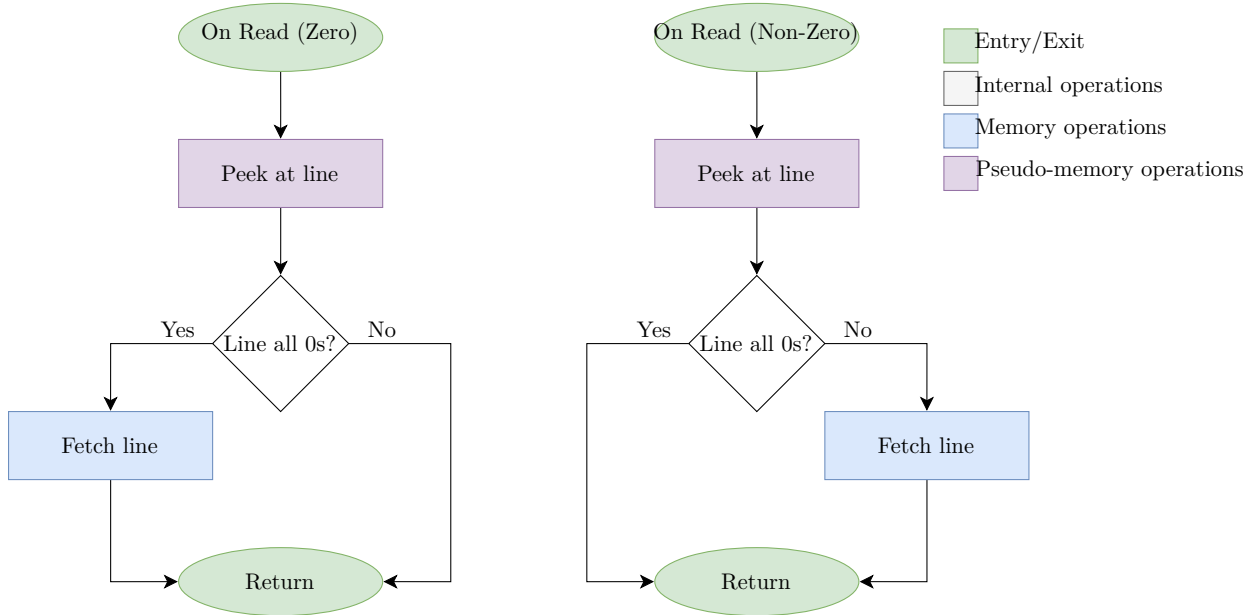


Figure 20: The Morello Project algorithm for reading tags. Pseudo-memory access operations are purple.

The algorithm for writing is as follows:

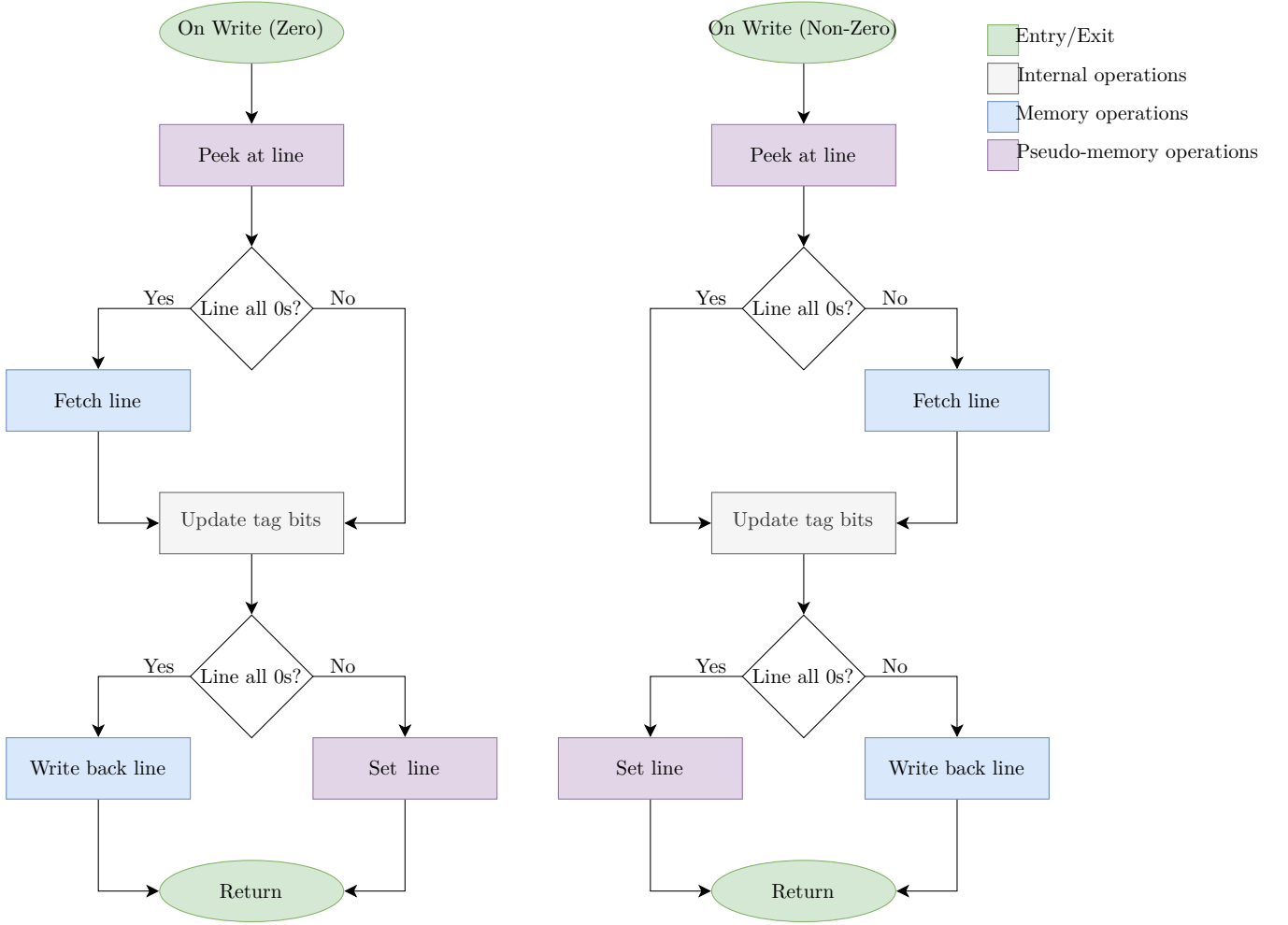


Figure 21: The Morello Project algorithm for writing tags.

The implementations of `handleRead` and `handleWrite` implement these algorithms.

3.2 Logging

3.2.1 Logging Tag Table State

Another job of the `Simulator` class is to stimulate the `Memory`, via the `Controller`'s proxy method, to dump its tag table state once a log point is reached.

When ChampSim parses our output trace, we want it to log the state of the tag cache at this same point of program execution. However, ChampSim's unit of work is a memory accesses, not an LLC miss. As there may be more than one memory access per the handling of a single LLC miss, we must let ChampSim know *after which memory access* (as opposed to after which LLC miss) it should dump its state. Therefore, after the `Simulator` stimulates the memory, it also reports the index of the most recent memory access request, which will later also be given to ChampSim for this purpose.

3.2.2 Adding Logging to ChampSim for Tag Cache State

To add cache state logging support to ChampSim, I had to modify three files.

In `cache.h` I simply made the `BLOCK` struct public so it could be accessed from outside of the class.

In `main.cc` I added to option to specify two extra command line arguments:

- the path of the output log file to log the tag cache state into
- the indices of the memory accesses after which a tag cache state log entry should be made

The `main` function passes this information to the `champsim::main` function defined in `champsim.cc`.

This function was then modified to check after each phase of processing if it has reached a log point. If it has, it dumps all of the addresses currently resident in the simulated cache into the output log file.

3.3 Verification

3.3.1 Microbenchmarks – Unit Tests

I implemented some unit tests to test the correctness of the address translations and access sequences of some of the concrete tag controller implementations. They have the controller handle a single access, and then analyse the sequence of addresses that the controller made in response, asserting that the addresses accessed match what was expected. These microbenchmarks were particularly useful to debug the controllers as I implemented them, as their precision highlighted exactly where the issues lay.

3.3.2 Macrobenchmark – Trace Verifier

The input trace files usefully contain information about the results of reads made to the DRAM tag table during the workload simulation. This allows us to cross-reference the results of reads that our implemented tag controllers produce with those found in the trace file. The trace files contain hundreds of millions (if not more) of read instructions, so if of all of the read results match, then we can say with extreme confidence that our implementation of the tag controller is correct. As part of my project I developed a program that allows the user to provide a reference workload and verify the correctness of a tag controller in this way.

Code Snippet 7 verify.cpp's verify function

```
1  int verify(Decoder &decoder, Controller &controller, size_t n) {
2      uint16_t tags, bufferTags;
3      int discrepancies = 0;
4
5      vector<memAccess> buffer(CHUNK_SIZE);
6      for (size_t at = 0; at < n; at += CHUNK_SIZE) {
7          size_t read = decoder.readLLCMisses(buffer, min(CHUNK_SIZE, n-at));
8
9          size_t i = 0;
10         for (size_t remaining = read; remaining > 0; --remaining) {
11             if (buffer[i].type == ACCESS_TYPE_READ) {
12                 tags = controller.handleRead(buffer[i]);
13                 bufferTags = buffer[i].tags;
14                 if (tags != bufferTags) {
15                     cout << "TAGS DIFFER AT INSTRUCTION INDEX " << i << endl;
16                     cout << "ADDRESS:          " << buffer[i].addr << endl;
17                     cout << "TRACE TAGS:      " << bufferTags << endl;
18                     cout << "BASELINE TAGS: " << tags << endl;
19                     discrepancies++;
20                 }
21             } else controller.handleWrite(buffer[i]);
22             ++i;
23         }
24     }
25     cout << "VERIFY: " << discrepancies << " total discrepancies" << endl;
26     return discrepancies;
27 }
```

3.3.3 Macrobenchmark – Cross-Verifier

The program also allows the user to specify a pair of tag controllers, and compare them against each other. It does this by taking an input workload, simulating both of them handling the desired number of LLC misses, and calling `handleRead` on every address in physical memory, ensuring that all tag state is functionally equivalent. As the controllers handle the LLC misses, their `Memorys`' tag table states will update according to the implementations of their algorithms. If at the end the states of all tags in the table match, the user can say with extreme confidence that the inputs, outputs, and logical state transitions of the two tag controller implementations are the same.

With this program, I also implemented two dummy controllers, `DummyZeroController` and `DummyOneController`, which always return zeroes or ones from their `handleRead` methods respectively. By comparing a scheme to either of these, a user can confirm whether or not their controller is only reading out zeroes or ones, which can be useful for debugging purposes.

Code Snippet 8 verify.cpp's crossVerify function

```
1  int crossVerify(Controller &controller1, Controller &controller2) {
2      memAccess ax;
3      uint16_t tags1, tags2;
4      int discrepancies = 0;
5
6      for (uint64_t addr = 0; addr < MEMORY_SIZE; addr += 16) {
7          ax = memAccess(ACCESS_TYPE_READ, 64, 0, addr);
8
9          tags1 = controller1.handleRead(ax);
10         tags2 = controller2.handleRead(ax);
11
12         if (tags1 != tags2) {
13             cout << "TAGS DIFFER AT ADDRESS " << addr << endl;
14             cout << "CONTROLLER 1 TAGS: " << tags1 << endl;
15             cout << "CONTROLLER 2 TAGS: " << tags2 << endl;
16             discrepancies++;
17         }
18     }
19     cout << "CROSS-VERIFY: " << discrepancies << " total discrepancies" << endl;
20     return discrepancies;
21 }
```

3.4 Visualisation Tool

With the ability to capture the state of the tag table and tag cache at any point in the execution of a workload, we can now create a tool to visualise this state, effectively showing us a ‘snapshot’ of the tag state at that moment in execution.

`visualiser.py` combines the simulation and the visualisation into one program. It allows the user to specify the following command line arguments:

- the name of the tag controller scheme to simulate
- the path of the LLC miss input trace file
- the path of the tag table initial state input file
- the maximum number of LLC misses to process
- the indices of the instructions after which a snapshot should be made
- the string prefix of the output image filenames

The following describes the program’s sequence of events:

1. The tag controller simulator is invoked, being passed the arguments it requires.
2. The simulation finishes, and the output files are produced.
3. The standard output of the tag controller simulation is read, collecting the reported memory accesses after which ChampSim must dump the state of the tag cache.

4. The instrumented version of ChampSim is invoked, being passed the required arguments.
5. ChampSim simulates the operation of the tag cache, and logs the state of the cache at the desired points. It then finishes, reporting its usual metrics to the user.
6. Now having the logged states of both the tag table and the tag cache, the program finally generates a PNG that visualises this state. One PNG is generated for each log point.

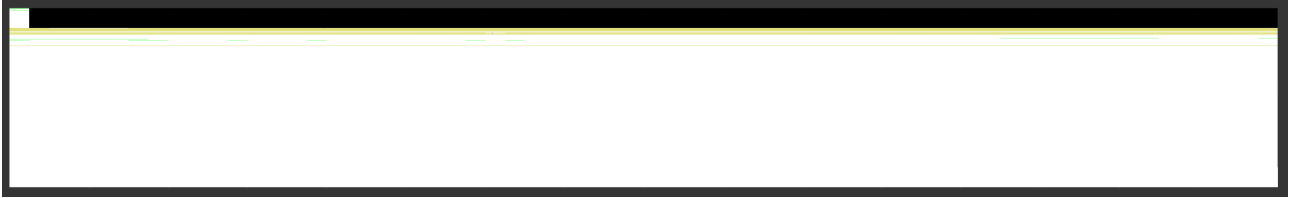


Figure 22: A snapshot of the tag table when running the Efficient Tagged Memory scheme.

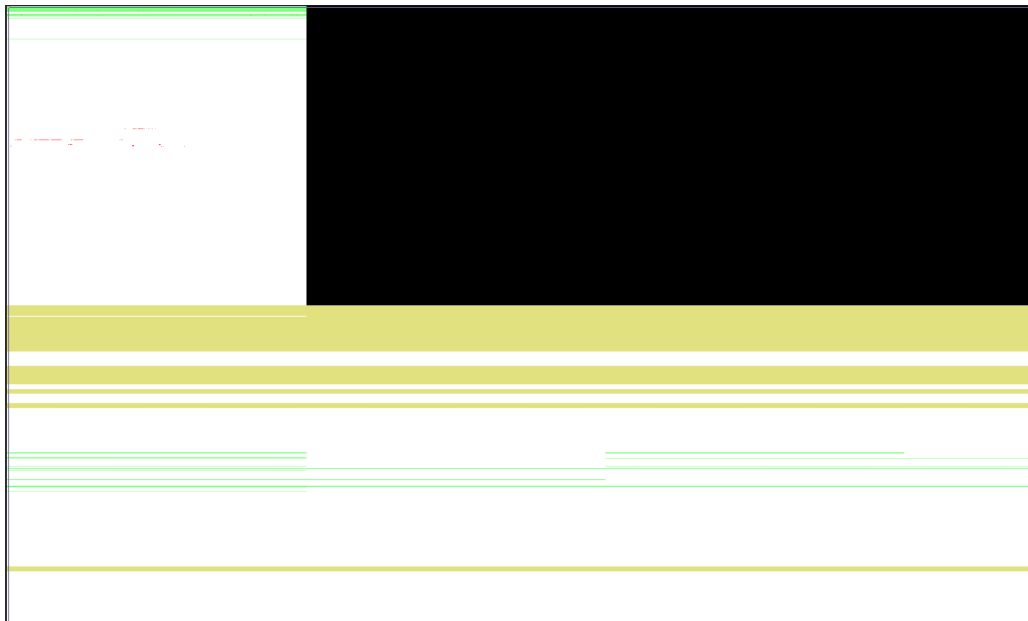


Figure 23: A closer look at the root table.

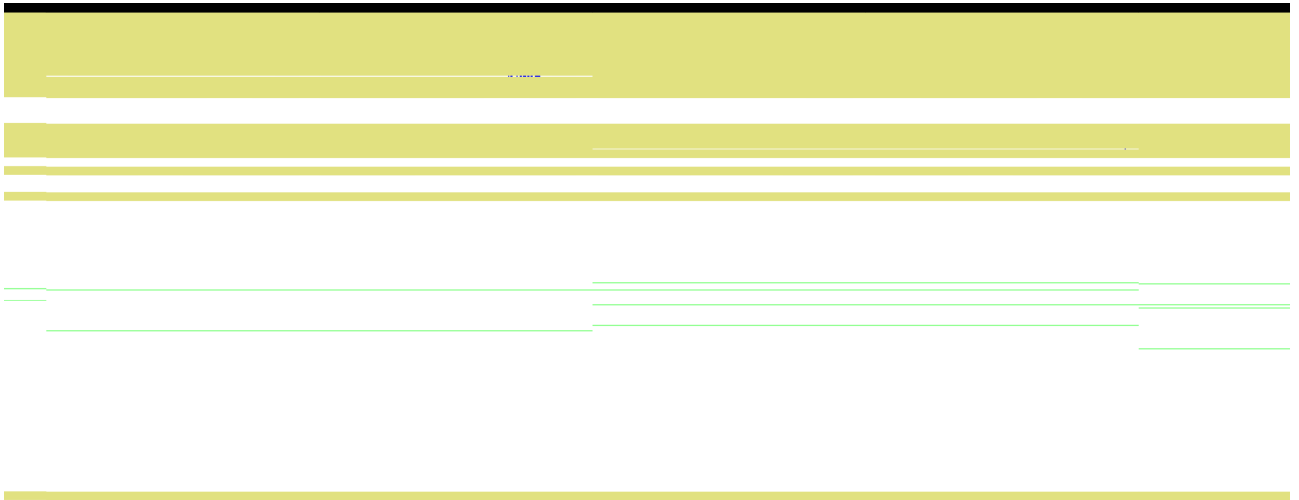


Figure 24: A closer look at the leaf table.

3.5 Phoenix – A Novel Scheme

3.5.1 The Motivation

After confirming the correctness of my implementations of the above three algorithms by using the verifiers, I used the visualisation tool to inspect the state of the tag table and cache during a simulation of the *Efficient Tagged Memory* scheme.

I noticed that while some tags were set, there were still huge portions of the table that contained no set tags. In a physical system, these correspond to kilobytes of memory that only contains zeroes. Seeing this suggested two things to me:

The first was that there might be a more efficient way to store all of these zeroes in the tag cache. *Efficient Tagged Memory*'s root table already does a very good job of representing the zeroes succinctly, but there may still be opportunities to increase the cacheability of the tag table in this respect.

The second was that there might be an opportunity to free these empty regions to a DRAM allocator, if the system were running under a hypervisor with such a mechanism. The above three schemes have no means for doing this.

3.5.2 The Intuition

3.5.2.1 Reference Counters

A thought that initially came to my mind to increase the cacheability of the tag table was to increase the scope of the root bits to more than just a single cacheline of leaf tags. Doing this would mean that we could contain a larger portion of the tag table in the tag cache, as each zero root bit would represent a larger section of the cache.

Increasing the scope significantly might also be a start to dynamic DRAM allocation. Current schemes have no notion of how much physical space they currently require for their tag tables, but A scheme that divides its tag table up into large blocks could allocate and free those blocks on-demand. If an entire block is unused, the space it is consuming could be freed back to the hypervisor's memory pool, to be used by another guest OS. Once the space is required again, the guest OS could request more physical memory.

Increasing the scope of the root bits brings problems, however. One problem is that on writing zeroes into the leaf table, we must check the entire group of tags covered by the corresponding root bit to see if all tags are now zero, so as we appropriately zero the root bit if so. A larger group means more cachelines must be retrieved during this process, increasing latency, polluting the cache, and increasing the chance of a cache miss.

A potential solution to this problem is to use a counter to keep track of the number of tags that are non-zero. Consider a scheme where each group of leaf tags has its own counter, and all of the counters are stored in a table. On setting tags, we increment the counter for that group, and on clearing them, we decrement it. Once the counter reaches zero, we know that all tags in the group must be zeroes, so we can clear the root bit without having to do additional checks. Lines from the counter table could also be held in the tag cache, consuming relatively small amounts of precious cache space, much like the root cachelines.

It is actually more appropriate to instead count the number of non-zero cachelines, instead of non-zero tags. This reduces how frequently we need to update the counter, helping to keep bus traffic down. Checking if the whole cacheline is non-zero instead of just the tag(s) the we update brings no additional memory accesses, as when serving requests to the LLC, we must access the entire cacheline anyway to fulfil the request.

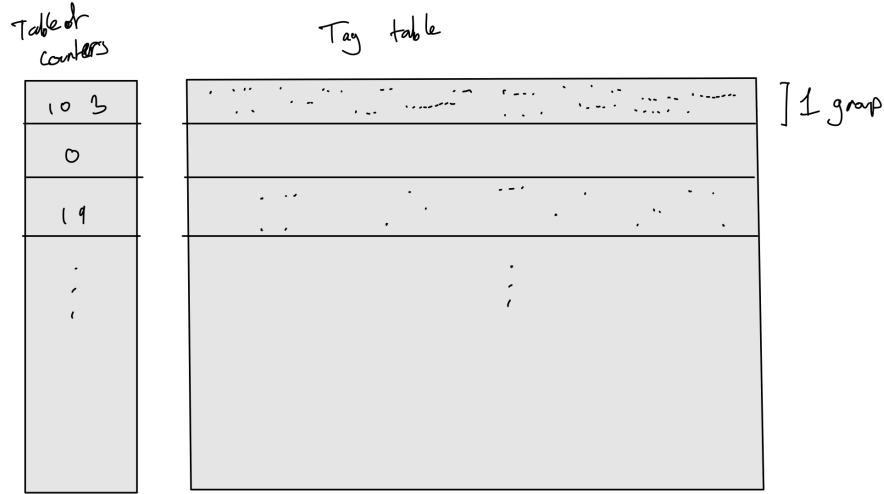


Figure 25: An illustration of the table of tag counters.

Suppose that we have M bytes of physical memory in which value data is stored. Let k be the number of tag cachelines in each counter group.

$$\frac{M \text{ value data bytes}}{16 \text{ value data bytes per tag bit}} = \frac{M}{16} \text{ tag bits} \quad (8)$$

$$\frac{\frac{M}{16} \text{ tag bits}}{512 \text{ tag bits per tag cacheline}} = \frac{M}{8192} \text{ tag cachelines} \quad (9)$$

$$\frac{\frac{M}{8192} \text{ tag cachelines}}{k \text{ tag cachelines per group}} = \frac{M}{8192k} \text{ tag groups, and counters} \quad (10)$$

Each counter will require at least $\text{ceil}(\lg(k+1))$ bits to keep track of it, as it must be able to represent all numbers in the range $[0, k]$.

Therefore the minimum extra space required by the table of counters will be $\frac{M}{8192k} \times \frac{\text{ceil}(\lg(k+1))}{8}$ bytes. In

practice, padding would be used to keep the counters from crossing word boundaries, consequently increasing the size of the table.

We must now choose an appropriate value for k , the group size. To do this, I used the visualisation tool to investigate the state of the tag table for some workloads. I noticed that the set tags tended to be focused into pages. That is, they tended to have either no tags set, or many tags set; relatively few pages had only a handful of tags set (see figure ??). This suggested to me that setting the group size to equal the size of a page of tags, 4096 bytes (64 cachelines), was a good choice. This way, the majority of the counters will be zero, and the rest will mostly be high numbers. The number is not too small so as to result in there being too many counters, requiring too much extra space and too much additional cache pollution. The number is also not too large so as to merge freeable regions of zeroes with regions containing tags, disallowing us from freeing the entire group. That is, due to the tags being concentrated amongst a small number of pages, we can still free most of the space being used to hold zero tags without having to use smaller granules.

Another advantage of choosing this number is that it is likely that the dynamic DRAM allocator operates on page-granularity, making the interface clean.

So, let $k = 64$. As an example, suppose that $M = 4\text{GiB}$:

We would have $\frac{2^{32}}{8192 \times 64} = 8192$ groups of tags, and also this many counters.

Each counter would require at least $\text{ceil}(\lg(64 + 1)) = 7$ bits to keep track of it.

Therefore the minimum extra space required by the table of counters will be $8192 \times \frac{7}{8} = 7186 = 7\text{KiB}$.

This is a somewhat significant space overhead, but it comes with the ability to now free pages to DRAM. By freeing only 2 pages, we can reduce the overhead to below that of the other schemes.

Regarding cache pollution, only the counters of groups currently in use will need to be kept in the cache, so the effect that the counters have on cache hit rate will be relatively small, but non-negligible.

3.5.2.2 The Superroot Table

I then iterated on this idea. The performance of the scheme detailed in *Efficient Tagged Memory* was thanks to its finely-tuned tag table. Ideally, we would not have to sacrifice this performance. However, another problem with increasing the scope of the root tags is that it also decreases the chance that all of the leaf tags it covers are zero, and thus we are more likely to need to perform a two-stage lookup, meaning we bring more zeroes into the tag cache than before. That is, the effective cacheability of the table has decreased – whenever we bring in a leaf line of zeroes, we risk evicting a root line, meaning the coverage of the tag cache is reduced. It is a balancing act to find the optimal scope for the root bits, which the *Efficient Tagged Memory* scheme has already calibrated for.

Instead of replacing the root table with a set of counters, we can extend *Efficient Tagged Memory*'s scheme, keeping the root table, but also maintaining page-wide counters to preserve the property of not needing to scan the entire page of leaf tags each time we clear one. If we were to keep the root table and also have counter groups that cover $k = 64$ tag cachelines, there would be $\frac{4096 \times 8}{512} = 64$ root bits for each counter.

By keeping the root table, we can now discard the counters entirely, replacing them with something more physically compact. We know that checking if all the tags in the page are cleared is equivalent to checking if all of the 64 corresponding root bits are clear, and as the 64 root bits fit in a word or two, this process is much more manageable than scanning an entire page, so we can simply do this instead of maintaining page-wide counters. Instead of counters, we can simply have a single bit that we clear whenever the corresponding 64 root bits are cleared, and we set when any of the 64 root bits are set. This effectively adds a third layer at the top of the hierarchical table. The “superroot” level has one bit for each page of leaf tags. These superroot bits are stored in their own table, beneath the root and leaf tables. By transitivity through the root level, we maintain the property that if any tags in the block are set, the superroot bit will be set, and it will be cleared otherwise.

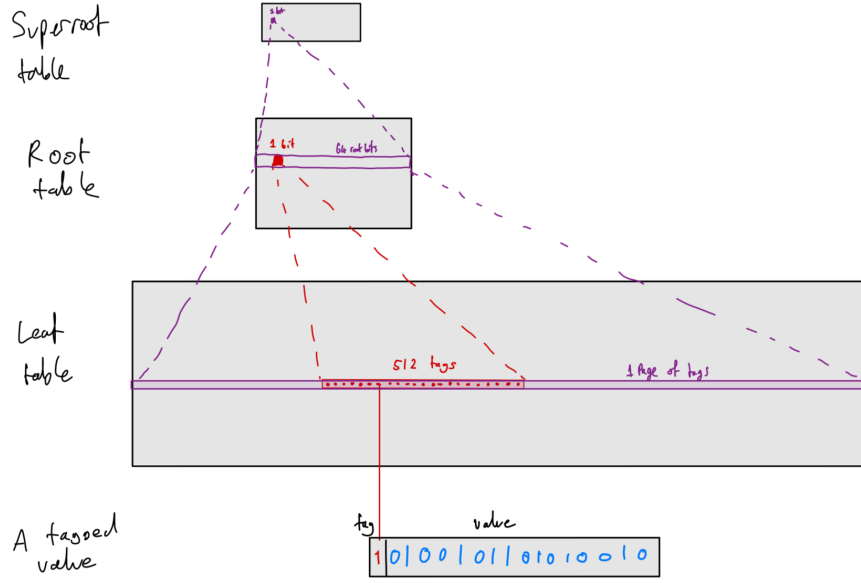


Figure 26: The Phoenix scheme’s three-level hierarchical lookup table.



Figure 27: The physical layout of the hierarchical table in the Phoenix scheme.

3.5.2.3 The Tag Working Set

In this scheme, we want to free pages of leaf tags that are unused and only contain zeroes. For pages that contain at least one leaf tag set, we must keep them allocated. To do this, we can mandate that whenever a superroot bit becomes zero, we free its corresponding page of leaf tags. On attempting to read the page, we will first consult the superroot bit, and upon noticing that it is zero, we will not attempt to read any deeper into the hierarchy, avoiding encountering a fault on attempting to read an unallocated page.

This would be correct, but it might result in undesirable thrashing of the DRAM allocator for some workloads – if a page only contains one tag that is constantly being set and unset, we would make many requests to the DRAM allocator to free and then re-allocate that page, likely resulting in increased latency experienced within our guest OS, and it has the potential to adversely affect the performance of other guest OSes if they happen to also be queueing to use the allocator.

To mitigate this, we have the concept of a “tag working set”. This is a set of pages in which the system is currently accessing *non-zero* tags. It operates similarly to a cache in the sense that it is a set which respects temporal locality: when we access non-zero tags in a page, we place that page into the tag working set, but

the set's finite size means we must in turn evict another page from the set, *ideally one for which we have not accessed non-zero tags for some time*.

The tag working set can help mitigate the thrashing problem. We can mandate that we only free a page once the following becomes true:

$$\text{The page contains no set tags} \wedge \text{The page is not in the tag working set} \quad (11)$$

When revisiting the thrashing example, we notice that we would not free the page immediately on clearing it, as it would still remain in the tag working set for some time. If we tune the parameters well, it will remain in the set until the tag is set once more, and we will then have avoided making any requests the allocator.

With this, we can describe a set of rules for when to update the tag working set, and when to allocate and free pages. Algorithm 9 details these rules.

Algorithm 9 Set of rules for when to update the tag working set, and when to allocate and free pages

```

1: On reading a non-zero tag from a page  $p$  do
2:   if  $p$  is in the tag working set then
3:     Promote  $p$  to the most-recently used in the tag working set;
4:   else
5:     Add  $p$  to the tag working set, marking it as the most-recently used;
6:   end if
7: End
8:
9: On writing a non-zero tag to a page  $p$  do
10:  if  $p$  is in the tag working set then
11:    Promote  $p$  to the most-recently used in the tag working set;
12:  else
13:    Allocate  $p$ ;
14:    Add  $p$  to the tag working set, marking it as the most-recently used;
15:  end if
16: End
17:
18: On clearing a page of tags  $p$  do
19:  if  $p$  is in the tag working set then
20:    Free  $p$ ;
21:  end if
22: End
23:
24: On Evicting a page  $p$  from the tag working set do
25:  if  $p$  contains no set tags then
26:    Free  $p$ ;
27:  end if
28: End

```

These rules maintain the following invariant:

$$\forall \text{ Tag Pages } . \text{Page is not allocated} \iff \text{Page is not in the tag working set} \wedge \text{Page has no tags set} \quad (12)$$

Which may be written equivalently as:

$$\forall \text{ Tag Pages } . \text{Page is allocated} \iff \text{Page is in the tag working set} \vee \text{Page has at least one tag set} \quad (13)$$

3.5.3 The Algorithm

The two subsections above describe the workings of the novel tag controller scheme that I devised. I named it the Phoenix scheme.

We now need an exact algorithm for reading from and writing to the tag table.

The following is the algorithm for reading:

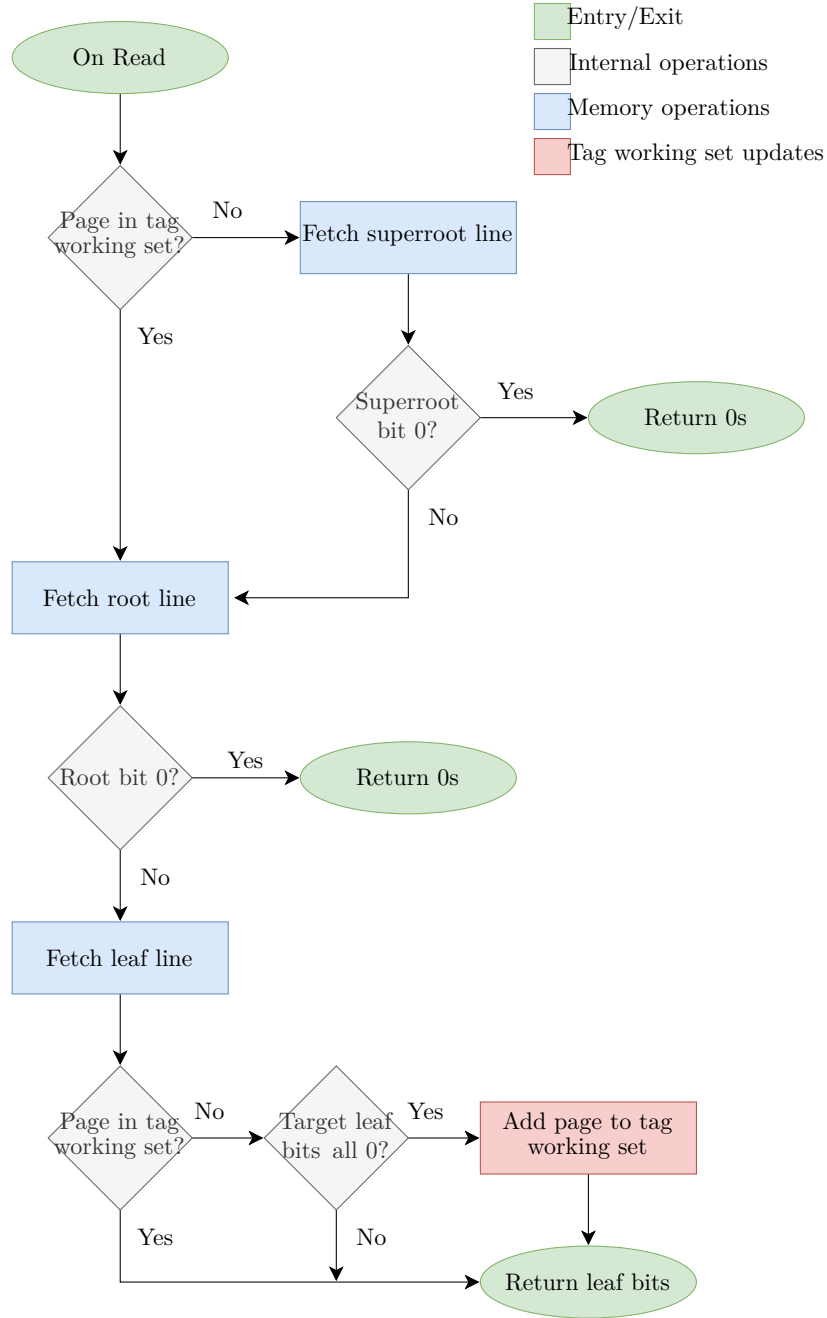


Figure 28: The Phoenix algorithm for reading tags. Tag working set operations are in red.

Introducing the tag working set has a secondary benefit. A design choice I made in the reading algorithm was that if the page we are reading is in the tag working set, we skip reading from the superroot table and instead read straight from the root table. Pages that are in the tag working set are most likely to have their superroot bits set, as after all it is *by accessing a set tag in the page* that a page enters the set. This way, we avoid the

extra third lookup for the cases where it is most likely that this lookup will not allow us to take a shortcut. As we skip straight to the root table, this means that in these cases the lookup speeds cannot be much worse than those experienced in the *Efficient Tagged Memory* scheme.

It is always safe to skip over the superroot bit when the page is in the tag working set; we never risk reading from an unallocated page of tags. Property 13 implies that the page must be allocated if the page is in the tag working set.

The *Efficient Tagged Memory* scheme uses a trick to avoid some unnecessary writes: when we clear a leaf cacheline, we need not write the cleared cacheline back into the leaf table, and instead we can simply clear the root bit, as when we next read these tags, even if their values in the table are stale, we will encounter the cleared root bit first, allowing us to take a shortcut to returning zeroes. This works because we always read the root before the leaf.

However, the previously mentioned design choice I made in the Phoenix scheme's read algorithm means we cannot use the same trick when we clear a group of root bits, as we do not always read the superroot before the root. This means that when a group of 64 root bits becomes clear, we must write them all back as zeroes, as well as updating the superroot bit. If we do not do this, then when we skip over the superroot table during a read, we might end up following a stale root table entry down into a stale leaf table cacheline, reading out 1s when we should be reading 0s. This can be seen in the algorithm for writing.

The following is the algorithm for writing:

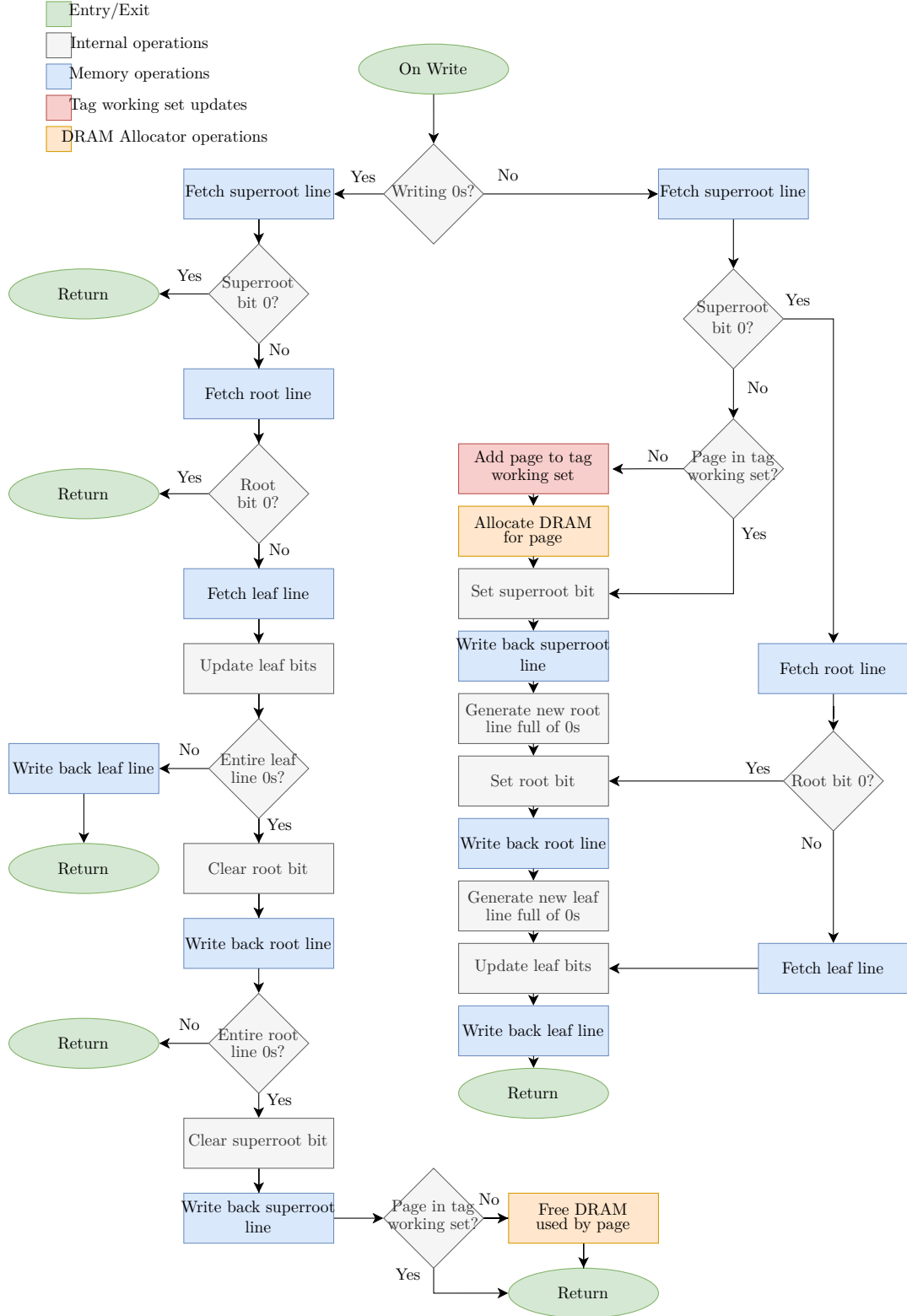


Figure 29: The Phoenix algorithm for writing tags. DRAM allocator operations are orange.

In the case where we update the tag working set and then immediately allocate a page in DRAM, there might be an opportunity to bypass the dynamic DRAM allocator. If the page we evict from the tag working set happens to contain no set tags, we would free it. Instead, as we are about to allocate a page, we might instead be able to simply transfer the page from one superroot bit to another, perhaps by changing mappings in the appropriate table. Whether or not this would be possible would depend on the interface of the DRAM allocator, and what is exposed to the guest OSes.

3.5.4 The Implementation

`PhoenixSController.h` defines the Phoenix scheme's tag controller.

On constructing an instance, two interesting parameters must be passed:

`twSize` is the maximum size of the tag working set. A smaller set would mean that on average fewer DRAM pages would be allocated, resulting in the average space overhead of the tag table decreasing. Also, lookup times and area consumption of the actual physical tag controller would decrease. A larger tag working set would provide greater locality benefits, and perhaps less allocator thrashing.

`useTrueLRU` defines whether we are simulating an exact LRU replacement scheme for the tag working set, or an approximation of it, namely NRU. If this parameter is specified as `false`, the NRU scheme used will be a worst-case NRU. That is, the second-most-recently used page will be evicted. This way, we can get an approximate lower bound on the performance of an actual implementation of non-true LRU.

The tag working set itself is implemented as a `std::vector` of page indices. Whenever we access non-zero tags in a page, we move the index of that page to the back of the vector if it is already in the set, or add it to the back if it is not.

Whenever we add a page, we must check that the size of the vector has not exceeded `twSize`. If it has, we choose our victim. If `useTrueLRU` is true, we pick the index at the front of the vector. Otherwise, we pick the index just before the back.

To check if an index is in the tag working set, we simply do a linear pass over the vector.

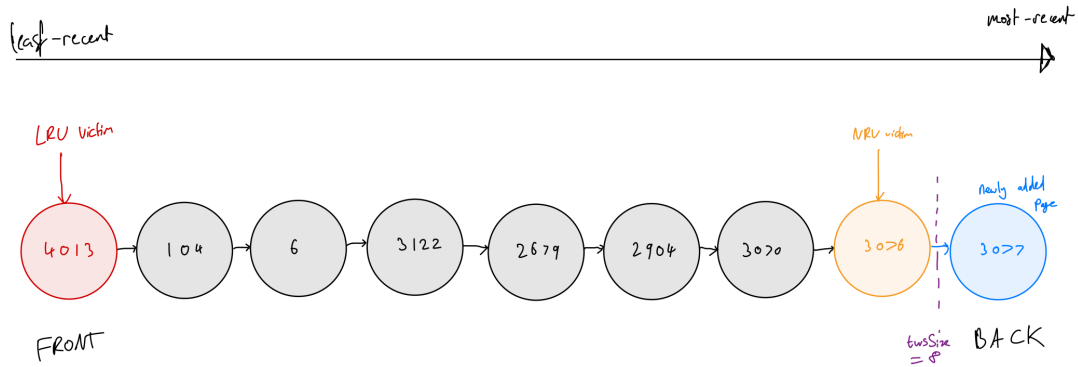


Figure 30: A visualisation of the implementation of the tag working set, and how the replacement policies choose their victims.

Realistically, the `twSize` will not be larger than a three-digit number, so the $O(n)$ cost that these operations have on the running time of the simulator is negligible.

We use a `std::unordered_set` to keep track of which pages are allocated in DRAM as the simulation progresses. The set is updated by the `DRAMAlloc` and `DRAMFree` methods, which are called by the `handleRead` and

`handleWrite` algorithms. It is also initialised during `setup`.

After each LLC miss that the controller serves, it records how many pages are currently allocated in DRAM – i.e. the size of the *unordered_set*. These sizes are stored in a frequency list, implemented as a `std::unordered_map` which maps a count to the frequency of that count. `reportStats` uses this frequency list to report some metrics at the end of the simulation.

3.6 Repository Overview

- the dirtree thing

4 Evaluation

5 Conclusions

References

- [1] Alexandre Joannou et al. “Efficient tagged memory”. In: *2017 IEEE International Conference on Computer Design (ICCD)*. IEEE. 2017, pp. 641–648.
- [2] Brian E. Clark and Michael J. Corrigan. “Application System/400 performance characteristics”. In: *IBM Systems Journal* 28.3 (1989), pp. 407–423.
- [3] Jedidiah R Crandall and Frederic T Chong. “Minos: Control data attack prevention orthogonal to memory model”. In: *37th International Symposium on Microarchitecture (MICRO-37’04)*. IEEE. 2004, pp. 221–232.
- [4] Joseph L Greathouse et al. “A case for unlimited watchpoints”. In: *ACM SIGPLAN Notices* 47.4 (2012), pp. 159–172.
- [5] G Edward Suh et al. “Secure program execution via dynamic information flow tracking”. In: *ACM Sigplan Notices* 39.11 (2004), pp. 85–96.
- [6] <https://www.cl.cam.ac.uk/research/security/ctsrd/cheri/>.
- [7] <https://www.morello-project.org/>.
- [8] <https://github.com/ChampSim/>.

Index (Optional)

Project Proposal